Universidad Rey Juan Carlos

ESCUELA DE INGENIERÍA DE FUENLABRADA

INGENIERÍA TELEMÁTICA

TRABAJO FIN DE GRADO

TÍTULO DEL TRABAJO CON LETRAS MAYÚSCULAS
PARA SUSTANTIVOS Y ADJETIVOS

Autor : Alejandro Tejero de la Morena

Tutor : Dr. David Moreno Lumbreras

Curso académico 2025/2026

©2025 Alejandro Tejero de la Morena

Algunos derechos reservados

Este documento se distribuye bajo la licencia

“Atribución-CompartirIgual 4.0 Internacional” de Creative Commons,

disponible en

https://creativecommons.org/licenses/by-sa/4.0/deed.es

Dedicado a

mi familia / mi abuelo / mi abuela

Agradecimientos

Aquí vienen los agradecimientos... Aunque está bien acordarse de la pareja, no hay que olvidarse de dar las gracias a tu madre, que aunque a veces no lo parezca disfrutará tanto de tus logros como tú... Además, la pareja quizás no sea para siempre, pero tu madre sí.

Resumen

Aquí viene un resumen del proyecto. Ha de constar de tres o cuatro párrafos, donde se presente de manera clara y concisa de qué va el proyecto. Han de quedar respondidas las siguientes preguntas:

- ¿De qué va este proyecto? ¿Cuál es su objetivo principal?
- ¿Cómo se ha realizado? ¿Qué tecnologías están involucradas?
- ¿En qué contexto se ha realizado el proyecto? ¿Es un proyecto dentro de un marco general?

Lo mejor es escribir el resumen al final.

Summary

Here comes a translation of the “Resumen” into English. Please, double check it for correct grammar and spelling. As it is the translation of the “Resumen”, which is supposed to be written at the end, this as well should be filled out just before submitting.

Índice general

1. Introducción **1**

1.1. Sección 1

1.1.1. Estilo 2

1.2. Estructura de la memoria 4

2. Objetivos **5**

2.1. Objetivo general 5

2.2. Objetivos específicos 5

2.3. Planificación temporal 5

3. Estado del arte **7**

3.1. Backend 7

3.1.1. Python 8

3.1.2. Django 9

3.1.3. Base de datos: de SQLite a PostgreSQL 10

3.2. Inteligencia artificial y modelos de lenguaje 10

3.2.1. Modelos de lenguaje y IA generativa 11

3.2.2. Ingeniería de prompts 11

3.2.3. Proveedores de modelos: OpenRouter y Groq 13

3.2.4. Formato OpenAI-compatible 14

3.3. Automatización e infraestructura 14

3.3.1. n8n 14

3.3.2. Docker 15

3.4. Frontend 16

3.4.1. HTML, CSS y JavaScript 17
3.4.2. Tailwind CSS 17

4. Diseño e implementación 19

4.1. Arquitectura general del proyecto 19
4.2. Fase 1: Definición del proyecto 21
4.3. Fase 2: Comienzo del desarrollo 22
4.3.1. Modelo de datos 23
4.3.2. Ingestion de datos 23
4.3.3. Deteccion de la colección principal 24
4.3.4. Sugerencia automatica 25
4.4. Fase 3: Integracion LLM y n8n 27
4.4.1. Conexion con LLM 27
4.4.2. Planner schema 28
4.4.3. Proteccion anti-alucinaciones 29
4.4.4. Actualizacion de modelos 31
4.4.5. Integracion de n8n 31
4.5. Fase 4: Generador de proyectos Django 32
4.5.1. Arquitectura del generador 32
4.5.2. Fallbacks generacion 34
4.5.3. Evolución del modelo de datos 34
4.5.4. Dockerfile, entrypoint y requirements 34
4.5.5. Workflow de despligue n8n 35
4.6. Fase 5: Refactor del proyecto y consolidación del generador 37
4.6.1. Refactor del generador en modulos 38
4.6.2. Refactor de templates y ficheros de estilos 39
4.6.3. Enriquecimiento de prompts 42
4.6.4. Trazabilidad: GenerationLog 42
4.6.5. Verificacion de consistencia 43
4.6.6. Panel de métricas 43
4.7. Fase 6: Funcionalidades avanzadas 44

4.7.1. Nuevos formatos de entrada: CSV y GeoJSON 44
4.7.2. Ejemplos few-shot por tipo de sitio 45
4.7.3. Versionado de proyectos generados 45
4.7.4. Usuarios del proyecto 46
4.7.5. LLM local y catalogo de modelos disponibles 46
4.8. Fase 7: Pulido y cierre 48
4.8.1. Sistema de presets visuales 48
4.8.2. Migración a PostgreSQL 49
4.8.3. Workflow generation-done 50
4.8.4. Workflow health-check 50
4.8.5. Workflow auto-shutdown 51
4.8.6. Mejora del workflow de despliegue 51

5. Experimentos y validación **53**

6. Resultados **55**

7. Conclusiones **57**
7.1. Consecución de objetivos 57
7.2. Aplicación de lo aprendido 57
7.3. Lecciones aprendidas 58
7.4. Trabajos futuros 58

A. Manual de usuario **59**

Bibliografía **61**

Índice de figuras

1.1. Página con enlaces a hilos 3
1.2. Estructura del parser básico 4

4.1. Arquitectura general del sistema WebBuilder 21
4.2. Sistema de scoring para la detección de la colección principal 25
4.3. Flujo de ingestión y análisis: de URL a schema 30
4.4. Los 7 pasos del generador de proyectos Django 33
4.5. Workflow de despliegue n8n 37
4.6. Estructura de módulos del generador tras el refactor 38

Capítulo 1

Introducción

En este capítulo se introduce el proyecto. Debería tener información general sobre el mismo, dando la información sobre el contexto en el que se ha desarrollado.

No te olvides de echarle un ojo a la página con los cinco errores de escritura más frecuentes[1].

Aconsejo a todo el mundo que mire y se inspire en memorias pasadas. Las memorias de los proyectos que he llevado yo están (casi) todas almacenadas en mi web del GSyC[2].

En mayo de 2023 me apunté a un curso de innovación docente donde nos pidieron hacer un podcast con temática docente. Aproveché entonces para hacer un podcast de unos 30 minutos donde en los primeros quince minutos introducía LaTeX y la memoria, y en los segundos hacía hincapién en aquellas cosas que más os cuestan utilizar en la memoria: las figuras, las tablas y las citas. Podéis escuchar el podcast en Internet[3].

1.1. Sección

Esto es una sección, que es una estructura menor que un capítulo.

Por cierto, a veces me comentáis que no os compila por las tildes. Eso es un problema de codificación. Al guardar el archivo, guardad la codificación de "ISO-Latin-1" a "UTF-8" (o viceversa) y funcionará.

[1] `http://www.tallerdeescritores.com/errores-de-escritura-frecuentes`

[2] `https://gsyc.urjc.es/~grex/pfcs/`

[3] `https://podcasters.spotify.com/pod/show/gregorio-robles9/episodes/Tu-memoria-de-Trabajo-Fin-de-Grado-o-de-Mster-en-LaTeX-e23hucr/a-a58kp2`

1.1.1. Estilo

Recomiendo leer los consejos prácticos sobre escribir documentos científicos en LaTeX de Diomidis Spinellis[4].

Lee sobre el uso de las comas[5]. Las comas en español no se ponen al tuntún. Y nunca, nunca entre el sujeto y el predicado (p.ej. en "Yo, hago el TFG" sobre la coma). La coma no debe separar el sujeto del predicado en una oración, pues se cortaría la secuencia natural del discurso. No se considera apropiado el uso de la llamada coma respiratoria o *coma criminal*. Solamente se suele escribir una coma para marcar el lugar que queda cuando omitimos el verbo de una oración, pero es un caso que se da de manera muy infrecuente al escribir un texto científico (p.ej. "El Real Madrid, campeón de Europa").

A continuación, viene una figura, la Figura 1.1. Observarás que el texto dentro de la referencia es el identificador de la figura (que se corresponden con el "label" dentro de la misma). También habrás tomado nota de cómo se ponen las "comillas dobles" para que se muestren correctamente. Nota que hay unas comillas de inicio (") y otras de cierre ("), y que son diferentes. Volviendo a las referencias, nota que al compilar, la primera vez se crea un diccionario con las referencias, y en la segunda compilación se "rellenan" estas referencias. Por eso hay que compilar dos veces tu memoria. Si no, no se crearán las referencias.

A continuación un bloque "verbatim", que se utiliza para mostrar texto tal cual. Se puede utilizar para ofrecer el contenido de correos electrónicos, código, entre otras cosas.

```
From gaurav at gold-solutions.co.uk  Fri Jan 14 14:51:11 2005
From: gaurav at gold-solutions.co.uk (gaurav_gold)
Date: Fri Jan 14 19:25:51 2005
Subject: [Mailman-Users] mailman issues
Message-ID: <003c01c4fa40$1d99b4c0$94592252@gaurav7klgnyif>

Dear Sir/Madam,
How can people reply to the mailing list?  How do i turn off
this feature? How can i also enable a feature where if someone
replies the newsletter the email gets deleted?
Thanks
```

[4] https://github.com/dspinellis/latex-advice

[5] http://narrativabreve.com/2015/02/opiniones-de-un-corrector-de-estilo-11-recetas-par
html

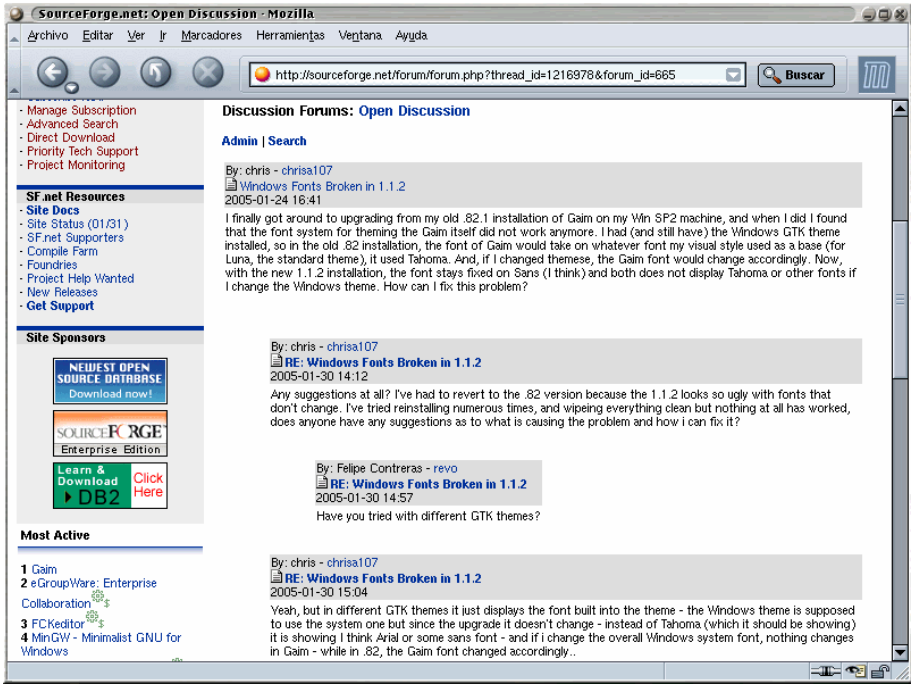

Figura 1.1: Página con enlaces a hilos

```
From msapiro at value.net  Fri Jan 14 19:48:51 2005
From: msapiro at value.net (Mark Sapiro)
Date: Fri Jan 14 19:49:04 2005
Subject: [Mailman-Users] mailman issues
In-Reply-To: <003c01c4fa40$1d99b4c0$94592252@gaurav7klgnyif>
Message-ID: <PC17302005011410485105780 1b04d55@msapiro>

gaurav_gold wrote:
>How can people reply to the mailing list?  How do i turn off
this feature? How can i also enable a feature where if someone
replies the newsletter the email gets deleted?

See the FAQ
>Mailman FAQ: http://www.python.org/cgi-bin/faqw-mm.py
article 3.11
```

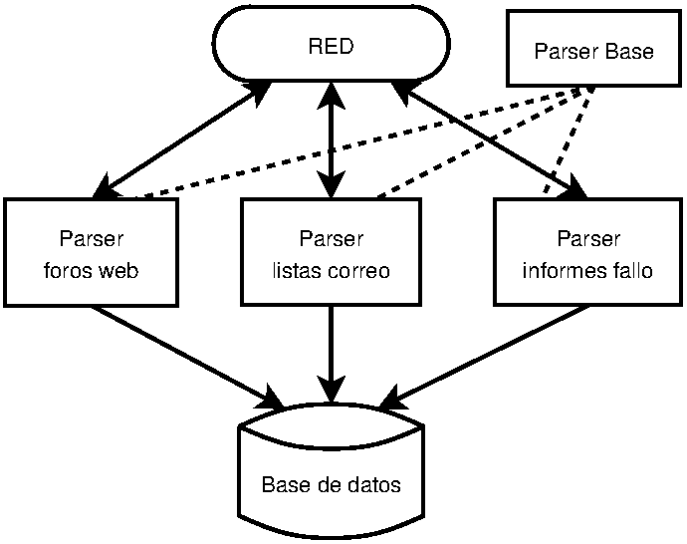

Figura 1.2: Estructura del parser básico

1.2. Estructura de la memoria

En esta sección se debería introducir la estructura de la memoria.

Así:

- En el primer capítulo se hace una intro al proyecto.
- En el capítulo 2 (ojo, otra referencia automática) se muestran los objetivos del proyecto.
- A continuación se presenta el estado del arte en el capítulo 3.
- ...

Capítulo 2

Objetivos

2.1. Objetivo general

Aquí vendría el objetivo general en una frase: Mi trabajo fin de grado consiste en crear de una herramienta de análisis de los comentarios jocosos en repositorios de software libre alojados en la plataforma GitHub.

Recuerda que los objetivos siempre vienen en infinitivo.

2.2. Objetivos específicos

Los objetivos específicos se pueden entender como las tareas en las que se ha desglosado el objetivo general. Y, sí, también vienen en infinitivo.

2.3. Planificación temporal

A mí me gusta que aquí pongáis una descripción de lo que os ha llevado realizar el trabajo. Hay gente que añade un diagrama de GANTT. Lo importante es que quede claro cuánto tiempo llevas (tiempo natural, p.ej., 6 meses) y a qué nivel de esfuerzo (p.ej., principalmente los fines de semana).

Capítulo 3

Estado del arte

En este capitulo describiré las tecnologías, herramientas y conceptos mas importantes que hicieron posible la creacion de WebBuilder, es decir, todo aquello que no ha sido creado en el marco de este trabajo pero que son fundamentales.

Cubriremos desde el framework web y la base de datos que sustentan la aplicacion, hasta los modelos de lenguaje y las herramientas de automatizacion e infraestructura que hacen posible el funcionamiento del proyecto.

El capítulo se organiza en cuatro bloques temáticos. En primer lugar se presentan las tecnologías de backend y base de datos que conforman el núcleo de la aplicación. A continuación se introduce el campo de la inteligencia artificial generativa y los modelos de lenguaje, pieza central del proyecto, junto con los conceptos de ingeniería de prompts y los proveedores de API utilizados. El tercer bloque cubre n8n, la herramienta de automatización que orquesta los flujos de notificación y despliegue. Por último, se describen Docker y Tailwind CSS, tecnologías de infraestructura y presentación empleadas en los proyectos web generados por la plataforma.

3.1. Backend

El backend esta construido con diferentes tecnologias de desarrollo web profesional. En esta seccion se describen las herramientas que conforman la capa de servidor y persistencia de datos de la aplicacion.

3.1.1. Python

Python es un lenguaje de programación interpretado, de alto nivel y útil para diferentes tipos de tareas. Creado por Guido van Rossum y publicado por primera vez en 1991, Python ha evolucionado hasta convertirse en uno de los lenguajes más utilizados del mundo, especialmente en el ámbito del desarrollo web, el análisis de datos y la inteligencia artificial. Siendo uno de los lenguajes de programacion mas versatiles.

La sintaxis clara permite escribir programas concisos para combinar logica del servidor, procesamiento de APIs y muchas cosas mas. Python cuenta con un ecosistema de bibliotecas muy completo, distribuido a través de PyPI (*Python Package Index*), del que WebBuilder hace uso directo: `requests` para la descarga de datos desde APIs externas, `xmltodict` y `defusedxml` para el parseo seguro de XML, y `zipfile` para el empaquetado de los proyectos generados, entre otras.

Durante todo el proyecto, mas alla del resto de tecnologias mencionadas a continuación, Python actua como base durante toda la logica del servidor, durante gran parte de procesos:

- **Ingestión de datos:** descarga y procesamiento del contenido devuelto por APIs externas en los formatos soportado por la plataforma (JSON, XML, CSV y GeoJSON).

- **Validación de datos:** comprobación del formato y la accesibilidad de la URL introducida por el usuario, así como la verificación de que los datos devueltos son parseables y contienen información útil.

- **Comunicación con modelos de lenguaje:** construcción y envío de los prompts al LLM, gestión de la respuesta y extracción del contenido estructurado necesario para cada fase del asistente.

- **Generación dinámica de código:** producción programática de todos los ficheros que componen el proyecto Django generado, incluyendo modelos, vistas, plantillas, configuración de despliegue y scripts de carga de datos.

La versión utilizada es Python 3, aprovechando características modernas del lenguaje como las *f-strings*, los gestores de contexto y el tipado opcional.

3.1.2. Django

Django es un framework de desarrollo web de codigo abierto para Python, que promueve el desarrollo rapido y escalado, asi como un diseño limpio. Fue publicado en 2005 y sigue el principio *"batteries included"*: incluye de serie todo lo necesario para construir aplicaciones web completas, desde el sistema de enrutado de URLs hasta el ORM (*Object-Relational Mapper*), el motor de plantillas, el sistema de autenticación y el panel de administración. Siendo desde el primer momento una web funcional y desplegable.

Permite configurar todo lo necesario, Django incluye un sistema de gestión de base de datos **SQLite** integrado que permite definir las tablas y sus campos directamente en Python, sin necesidad de escribir SQL. Cada tabla de la base de datos se representa como una clase, y Django se encarga de traducir automáticamente las operaciones sobre esos objetos (consultas, inserciones, actualizaciones) a las instrucciones SQL correspondientes. Esto facilita el desarrollo inicial, y tambien la evolucion del modelo a través de migraciones automáticas, que registran cada cambio de forma controlada, siendo capaz de saber que se cambio o modifico en cada momento, teniendo mas control sobre el desarrollo.

La logica de vistas y de URLs permite organizar la logica de forma clara y separada, con funciones clave que responden peticiones HHTP, separando el procesamiento de datos de la presentacion. El motor de plantillas permite generar HTML dinámico con una sintaxis sencilla, con soporte para herencia de plantillas, lo que favorece la reutilización de componentes visuales.

En el proyecto, Django gestiona el ciclo completo de cada ejecución:

En WebBuilder, Django gestiona el ciclo de vida completo de cada análisis:

- **Recepción de la URL:** recibe y valida la URL de la API introducida por el usuario a través del asistente.
- **Ingestión y parseo:** coordina la descarga del contenido de la API y su procesamiento en el formato correspondiente (JSON, XML, CSV o GeoJSON).
- **Invocación del módulo LLM:** envía los datos procesados al cliente de lenguaje y gestiona la respuesta para cada fase del asistente.
- **Persistencia:** almacena en base de datos el estado del análisis, los resultados obtenidos y el historial de generaciones por usuario.

- **Interfaz de usuario:** sirve las vistas del asistente paso a paso, el historial, las métricas y las páginas de administración.

La versión empleada es Django 5.1.

3.1.3. Base de datos: de SQLite a PostgreSQL

La persistencia de datos ha ido evolucionando a lo largo de todo el proyecto, en las fases iniciales se utilizó SQLite, que es la base de datos que Django configura por defecto. SQLite almacena toda la base en un unico fichero local, lo que simplifica enormemente la puesta en marcha del proyecto, asi como las multiples prueba, sin necesidad de instalar ni configurar ningun servidor externo.

Sin embargo, esta base de datos esta mas relacionada con pruebas y con el despligue real del proyecto, presentando algunas limitaciones en escrituras concurrentes, ya que utiliza bloqueos en el fichero de datos. A medida que WebBuilder se fue desarrollando e incorporando funcionalidades complejas como el registro de multiples generaciones simultaneas, el historial por usuario o el panel de metricas, estas limitaciones se fueron haciendo mas evidentes.

Por este motivo, en las fases finales del proyecto, cuando se realizaban pruebas continuas para el testeo del flujo, se realizó la migración a PostgreSQL. Este es un sistema gestor de bases de datos relacional de código abierto, robusto y ampliamente utilizado en entornos de producción. PostgreSQL ofrece soporte completo para transacciones ACID, concurrencia real mediante MVCC (*Multi-Version Concurrency Control*) y un rico conjunto de tipos de datos. Gracias a la capa de abstracción que proporciona el ORM de Django, esta migración no requirió modificar la lógica de la aplicación, sino únicamente actualizar la configuración de conexión y ejecutar las migraciones existentes sobre el nuevo motor.

3.2. Inteligencia artificial y modelos de lenguaje

La inteligencia artificial generativa constituye el nucleo funcional del proyecto. En esta seccion se introducen los conceptos fundamentales sobre los modelos de lenguaje, el prompting engenieering y los proveedores de API uilizados para acceder a los diferentes modelos.

3.2.1. Modelos de lenguaje y IA generativa

Los modelos de lenguaje de gran tamaño (LLM, *Large Language Models*) son sistemas de inteligencia artificial entrenados sobre grandes volumes de datos, a forma de entramiento y aprendizaje de los mismos. Su arquitectura se basa en redes neuronales de tipo *Transformer*, que utilizan un mecanismo de atención para capturar relaciones de largo alcance entre palabras.

A diferencia de los sistemas tradicionales de procesamiento de lenguaje natural, los LLM no se programan con reglas explícitas, sino que aprenden a predecir la siguiente palabra (o *token*) de una secuencia a partir de las anteriores. Este simple objetivo, aplicado a escala masiva, da lugar a modelos capaces de mantener conversaciones, redactar textos, traducir idiomas, resumir documentos o, como en este caso, capaces de generar código funcional a partir de instrucciones en lenguaje natural.

La irrupción mediática de modelos como ChatGPT en 2022 marcó el inicio de una etapa de adopción masiva de la IA generativa. Desde entonces, existen muchos modelos mejorados respecto al incio, tanto modelos comerciales (GPT-4, Claude, Gemini) como los modelos de pesos abiertos (LLaMA, Mistral, DeepSeek), accesibles a través de APIs públicas o ejecutables localmente. Esta diversidad y sobre todo a la existencia de modelos gratuitos permite a WebBuilder a elegir el modelo más adecuado para cada tarea en función de criterios como coste, latencia o calidad de respuesta.

En este trabajo, los LLM no se utilizan como simples chatbots, sino como componentes programables dentro de un flujo automatizado: el modelo recibe datos estructurados, devuelve análisis razonados y produce código fuente que se integra directamente en el proyecto generado.

3.2.2. Ingeniería de prompts

La ingeniería de prompts (*prompt engineering*) es la disciplina que estudia cómo formular las instrucciones enviadas a un modelo de lenguaje para obtener respuestas precisas y acorde a una necesidad, intentando a su vez reducir el numero de alucionaciones. A diferencia de la programación tradicional, donde el comportamiento del sistema se especifica mediante código, en los LLM el comportamiento se controla a través del lenguaje natural del propio prompt.

Para poder diseñar un buen prompt debemos seguir y combinar varios elementos:

- **Rol del modelo:** una descripción clara de la identidad o función que debe asumir el

modelo durante la tarea.

- **Instrucciones detalladas:** una especificación precisa de lo que se espera que el modelo haga y cómo debe hacerlo.

- **Formato de respuesta:** indicaciones explícitas sobre la estructura que debe tener la salida, como JSON, código Python o texto plano.

- **Restricciones:** limitaciones explícitas que acotan el comportamiento del modelo y reducen la probabilidad de respuestas incorrectas o fuera de contexto.

- **Ejemplos de referencia:** muestras del estilo de salida esperado que guían al modelo hacia el resultado deseado. Esta técnica, conocida como *few-shot prompting*, ha demostrado mejorar notablemente la calidad de las respuestas en tareas complejas o con formatos estrictos [1].

La calidad del resultado generado por un LLM depende, en **gran medida**, de la precisión del prompt. Un prompt ambiguo puede provocar respuestas inconsistentes, alucinaciones o salidas que no respetan el formato esperado. Por ello, el diseño, la iteración y el refinamiento de los prompts constituye una parte central del trabajo de desarrollo en sistemas basados en LLM, comparable en importancia a la propia escritura del código que los invoca.

Como comenté en el parrafo anterior, la calida de la generación depende en gran medida del promtp, pero una parte tambien importante es la potencia o calidad del LLM con el que estemos trabajando. Si el LLM no tiene potencia o esta poco entrenado, aun teniendo un buen prompt este modelo no tendrá suficiente contenido como para generar la respuesta que buscamos.

En el contexto de WebBuilder, la ingeniería de prompts juega un papel crítico en dos fases: el análisis de los datos de la API y la generación del código Django del proyecto. Cada uno de estos prompts ha sido diseñado tras innumerables pruebas para producir resultados coherentes y de calidad profesional, como se detallará en el capítulo de diseño e implementación. Ademas, durante el proyecto se fueron guardando los datos de cada generacion para realizar un estudio exhaustivo de que funciona mejor para cada caso.

3.2.3. Proveedores de modelos: OpenRouter y Groq

Para acceder a un LLm y poder usarlo, en la mayoria de casos es necesario recurrir a un proveedor de API que disponga del modelo y ofrezca su uso. Cada vez existen mas proveedores en el mercado, cada uno con su propio catalogo. La gran mayoria de estos proveedores siguen el estandar de OpenAI, lo que permite conectar de forma sencilla con WebBuilder. Cabe aclarar que los modelos gratuitos de los proveedores mencionados a continuacion realizan bien su trabajo, pero nunca de forma perfecta. Las alucinaciones a dia de hoy siguen existiendo sobre todo en los modelos gratuitos.

Groq es un proveedor especializado en inferencia de baja latencia. Utiliza hardware propio (LPU, *Language Processing Units*) optimizado, lo que le permite ofrecer velocidades de generación muy superiores a las de proveedores tradicionales basados en GPU. Groq aloja modelos de pesos abiertos como LLaMA, y dispone de un nivel de uso gratuito con límites de peticiones por minuto y por dia. Su rapidez hace que sea el mejor para WebBuilder, donde la generacion de templates es rapidisima.

OpenRouter es un proveedor que da acceso a decenas de modelos de distintos proveedores a través de una única API. Lo mejor de este proveedor es la sencillez para la conexion: una sola clave de API permite consultar modelos de OpenAI, Anthropic, Google, Meta o cualquier otro proveedor integrado en la plataforma. Esto facilita las pruebas comparativas entre modelos y reduce el coste de cambiar de proveedor en caso necesario.

NVIDIA NIM (*NVIDIA Inference Microservices*) es el catálogo de modelos e inferencia en la nube de NVIDIA. Ofrece acceso a más de cien modelos de distintos fabricantes, entre ellos Meta, Mistral, DeepSeek y los propios modelos Nemotron de NVIDIA,ejecutados sobre infraestructura de GPUs de alto rendimiento. Al igual que los proveedores anteriores, expone sus modelos a través de una API compatible con el formato OpenAI, lo que permite integrarlo en WebBuilder sin ninguna modificación en el cliente LLM. Su inclusión en el catálogo de proveedores de la plataforma amplía las opciones disponibles para el usuario, que puede elegir entre modelos optimizados para generación de código, razonamiento o tareas de propósito general.

3.2.4. Formato OpenAI-compatible

Este estándar ha sido adoptado por la práctica totalidad de los proveedores de LLM relevantes, lo que permite a las aplicaciones cliente comunicarse con cualquiera de ellos cambiando únicamente la URL base, la clave de API y el identificador del modelo. De esta forma el sistema puede operar indistintamente con Groq, OpenRouter, Nvidia o cualquier otro proveedor compatible, sin necesidad de adaptar la lógica interna, simplemente modificando esas variables.

3.3. Automatización e infraestructura

Como se comento anteriorme, WebBuilder no solo se apoya en el backend de Django y Pyhton, tambien se usan herramientas extenas para la automatizacion de tareas y el despliegue de los proyectos. En esta sección se describen las herramientas de n8n, encargado de orquestar los flujos asíncronos de la plataforma, y de Docker, que proporciona el entorno aislado en el que se ejecutan los proyectos generados.

3.3.1. n8n

n8n es una plataforma de automatización de flujos de trabajo de código abierto que permite conectar aplicaciones, APIs y servicios mediante una interfaz visual basada en nodos. Cada flujo de trabajo (*workflow*) está compuesto por una secuencia de nodos quese ejecutan en cadena, donde cada nodo representa una accion, explicando su funcion dentro del contenido del mismo nodo, hay varias funciones:

- **Disparadores:** eventos que inician el flujo, como una petición HTTP entrante (webhook) o un temporizador programado (cron).
- **Peticiones HTTP:** llamadas a APIs externas o internas para intercambiar datos entre servicios.
- **Transformación de datos:** procesamiento y manipulación de la información entre nodos mediante lógica personalizada en JavaScript.
- **Envío de correos:** notificaciones automáticas por email a usuarios o administradores ante determinados eventos del sistema.

- **Ejecución de scripts:** bloques de código personalizado que permiten implementar lógica compleja directamente dentro del flujo.
- **Consultas a base de datos:** lectura o escritura de datos en sistemas de persistencia externos integrados en el workflow.

Los flujos pueden pueden iniciarse a partir de webhooks (peticiones HTTP entrantes), disparadores temporales (*cron*) o eventos de aplicaciones conectadas, y muchos de los nodos implementados en este proyecto son de logica de programacion. Estos nodos permite ejecutar fragmentos de codigo JavaScript personalizado, implementando logicas completas como la limpieza y despliegue de contenedores Docker.

En WebBuilder, n8n actúa como una capa desacoplada que gestiona todo lo que no forma parte del ciclo síncrono de petición-respuesta de Django:

n8n actúa como una capa desacoplada que gestiona todo lo que no forma parte del ciclo síncrono de petición-respuesta de Django: el envío de correos automáticos al registrarse o iniciar sesión, la notificación al finalizar una generación, la monitorización diaria del estado del sistema, el apagado automático de contenedores inactivos y, sobre todo, el despliegue de los proyectos generados en contenedores Docker. Esta arquitectura permite añadir nuevas automatizaciones sin modificar el código del backend principal.

A diferencia de plataformas similares, n8n puede autoalojarse, lo que permite ejecutarlo en infraestructura propia y mantener el control completo sobre los datos y las credenciales. Como ocurre en este caso que esta desplegado dentro de un contenedor en local, pudiendo acceder y tener el control total de la herramienta

3.3.2. Docker

Dcoker es una herramienta que permite empaquetar aplicaciones junto con sus dependencias en unidades aisladas llamadas contenedores. A diferencia de la virtualización tradicional, los contenedores comparten el núcleo del sistema operativo anfitrión, lo que los hace mucho más ligeros y rápidos de arrancar que una máquina virtual convencional.

Cada contenedor se construye a partir de una imagen, un artefacto inmutable que contiene el sistema de ficheros, el software instalado y la configuración necesaria para ejecutar la aplicación. Las imágenes se definen mediante un fichero de texto llamado `Dockerfile`, que

describe paso a paso cómo construir el entorno: la imagen base, los paquetes a instalar, el código a copiar y el comando que se ejecuta al arrancar el contenedor.

Esta capacidad de empaquetado garantiza que la aplicación se ejecute de forma idéntica en cualquier entorno: el equipo de desarrollo, un servidor de pruebas o un entorno de producción. Esto resuelve uno de los problemas clásicos del despliegue, ejecutandose correctamente en algunas maquinas pero en otras no, debido a las diferencias entre los entornos en los que se desarrolla y se ejecuta una aplicación.

En WebBuilder, Docker cumple dos funciones diferenciadas. Por un lado, aloja la propia herramienta de n8n utilizada por la plataforma de forma local. Por otro, y más importante, cada proyecto Django generado por WebBuilder incluye automáticamente su propio `Dockerfile`, lo que permite desplegarlo en un contenedor aislado sin necesidad de instalar dependencias en el sistema anfitrión. Un flujo especifico de n8n (webbuilder-deploy), construye la imagen, lanza el contenedor y le asigna un puerto disponible de forma totalmente automática.

3.4. Frontend

WebBuilder tiene dos capas de frontend diferenciadas: la interfaz de la propia plataforma, construida con tecnologías web estándar, y los proyectos que genera.

En este primer caso, la interfaz de WebBuilder está construida con HTML, CSS y JavaScript, las tres tecnologías fundamentales de la web, estandarizadas por el W3C (*World Wide Web Consortium*). HTML define la estructura de cada página, CSS controla su presentación visual y JavaScript gestiona la interactividad en el lado del cliente. En WebBuilder estas tecnologías se utilizan en las plantillas Django para construir la interfaz del asistente paso a paso, el historial de análisis, el visor de código generado y el panel de métricas. JavaScript juega un papel especialmente relevante en aquellas vistas donde se requiere dinamismo sin recargar la página, como la previsualización del plan propuesto por el LLM o el editor de código integrado.

Además, durante el flujo de generación, los proyectos web que WebBuilder produce incluyen una capa de presentación basada en tecnologías modernas de frontend. A diferencia de la interfaz principal, estos proyectos utilizan Tailwind CSS como sistema de estilos, un framework que encaja especialmente bien con la generación automática de interfaces por parte de modelos de lenguaje, como se describe proximamente.

3.4.1. HTML, CSS y JavaScript

HTML (*HyperText Markup Language*) es el lenguaje de marcado que define la estructura y el contenido de las páginas web. En WebBuilder, todas las plantillas Django están escritas en HTML, organizando los elementos visuales del asistente, los formularios, las tablas de historial y los paneles de información.

CSS (*Cascading Style Sheets*) es el lenguaje de estilos que controla la presentación visual del HTML: colores, tipografías, espaciados, disposición de los elementos y diseño responsive. La interfaz de WebBuilder utiliza CSS para definir tanto el tema visual general de la plataforma como los modos claro y oscuro implementados durante el desarrollo.

JavaScript es el lenguaje de programación que añade interactividad en el lado del cliente, permitiendo modificar el contenido de la página sin necesidad de recargarla. En WebBuilder se emplea en varias vistas clave: la previsualización dinámica del plan propuesto por el LLM, el editor de código integrado para inspeccionar los ficheros generados y la actualización del estado del asistente en cada paso del flujo.

3.4.2. Tailwind CSS

Tailwind CSS es un framework de hojas de estilo basado en el enfoque *utility-first*. A diferencia de frameworks tradicionales como Bootstrap, que ofrecen componentes predefinidos (botones, tarjetas, formularios) listos para usar, Tailwind proporciona un conjunto extenso de clases que se combinan directamente en el HTML para construir cualquier diseño desde cero.

Este enfoque ofrece varias ventajas prácticas. En primer lugar, evita el problema de los estilos huérfanos y la acumulación de CSS no utilizado, ya que cada clase aplicada está explícitamente presente en el marcado. En segundo lugar, permite un control fino sobre el diseño sin necesidad de sobrescribir estilos predefinidos, lo que reduce la complejidad del CSS final. Por último, su sistema de diseño coherente con escalas predefinidas para espaciados, colores, tipografías y puntos de ruptura responsivos favorece la consistencia visual entre componentes.

Tailwind se ha convertido en una de las opciones más populares para proyectos web modernos, especialmente en aquellos generados a partir de plantillas o asistentes automáticos, ya que su naturaleza declarativa encaja particularmente bien con la generación de interfaces por parte de modelos de lenguaje. Los LLM tienden a producir HTML con clases Tailwind de forma

natural, gracias a la abundancia de ejemplos presentes en sus datos de entrenamiento.

En WebBuilder, todos los proyectos Django generados incluyen Tailwind CSS como sistema de estilos por defecto. Las plantillas HTML producidas por el LLM utilizan exclusivamente clases utilitarias de Tailwind, lo que garantiza una apariencia profesional y consistente sin necesidad de mantener hojas de estilo personalizadas para cada proyecto generado.

Capítulo 4

Diseño e implementación

4.1. Arquitectura general del proyecto

Antes comenzar a detallar los diferentes sprints donde se explica el desarrollo de cada avance, se presenta la arquitectura final del sistema, para que el lector pueda entender el flujo completo y como fue desarrollandose.

Como se muestra en la Figura 4.1, el sistema se divide en tres bloques principales.

El primero es **Django**, que actúa como núcleo de la aplicación. Dentro de este bloque, el módulo de ingestión se encarga de validar la URL introducida por el usuario,descargar el contenido de la API de forma segura y detectar automáticamente el formato de los datos, ya sea JSON, XML, CSV o GeoJSON. A continuación, el módulo de análisis LLM procesa la estructura del dataset y propone un schema con los campos más relevantes,que el usuario puede revisar y modificar antes de continuar. Una vez aceptado el plan, el generador produce un proyecto Django completo en siete pasos secuenciales, generando desde los modelos y vistas hasta los templates HTML y los ficheros de infraestructura necesarios para el despliegue. Todo el proceso queda registrado en la base de datos PostgreSQL, que almacena tanto los modelos de la aplicación como los logs de cada llamada al LLM para facilitar la depuración. Finalmente, el módulo de notificaciones informa al usuario en los momentos clave del flujo: al registrarse, al iniciar sesión y cuando su sitio ha sido generado correctamente.

El segundo es **n8n**, que gestiona las automatizaciones más costosas en tiempo y que no pueden ejecutarse de forma síncrona desde Django. En total se desarrollaron seis workflows, agrupados en dos categorías. Por un lado, los workflows de **autenticación y notificación**: *Login*

y *Register* gestionan el envío de correos al usuario al iniciar sesión o registrarse, y *generation-done* notifica al usuario cuando su sitio ha sido generado correctamente. Por otro lado, los workflows de **infraestructura**: *deploy* recibe la señal de Django mediante un webhook, construye la imagen Docker del proyecto generado, arranca el contenedor en un puerto libre y verifica que el sitio ha quedado operativo antes de devolver la URL de preview; *health-check* se ejecuta de forma programada comprobando diariamente que el servidor sigue activo y alertando al administrador si detecta algún problema; y *auto-shutdown* apaga automáticamente los contenedores que llevan inactivos más de un tiempo configurable, liberando recursos del servidor sin intervención manual.

El tercero son los **servicios externos**, que el sistema consume pero no controla. El proveedor de LLM, ya sea Groq, OpenRouter o un modelo configurado por el propio usuario, recibe los prompts del módulo de análisis y del generador y devuelve las respuestas en texto plano. Las APIs públicas son la fuente de datos que el usuario introduce en el sistema y de las que Django descarga el contenido a procesar. El servidor de correo recibe las notificaciones de n8n y las entrega al usuario final. Por último, el host Docker es el entorno donde se ejecutan los contenedores de los sitios generados, accesibles mediante una URL de preview.

La comunicación entre Django y n8n se realiza exclusivamente mediante webhooks, de forma que ambos sistemas están desacoplados y un fallo en las automatizaciones nunca bloquea el funcionamiento principal de la aplicación.

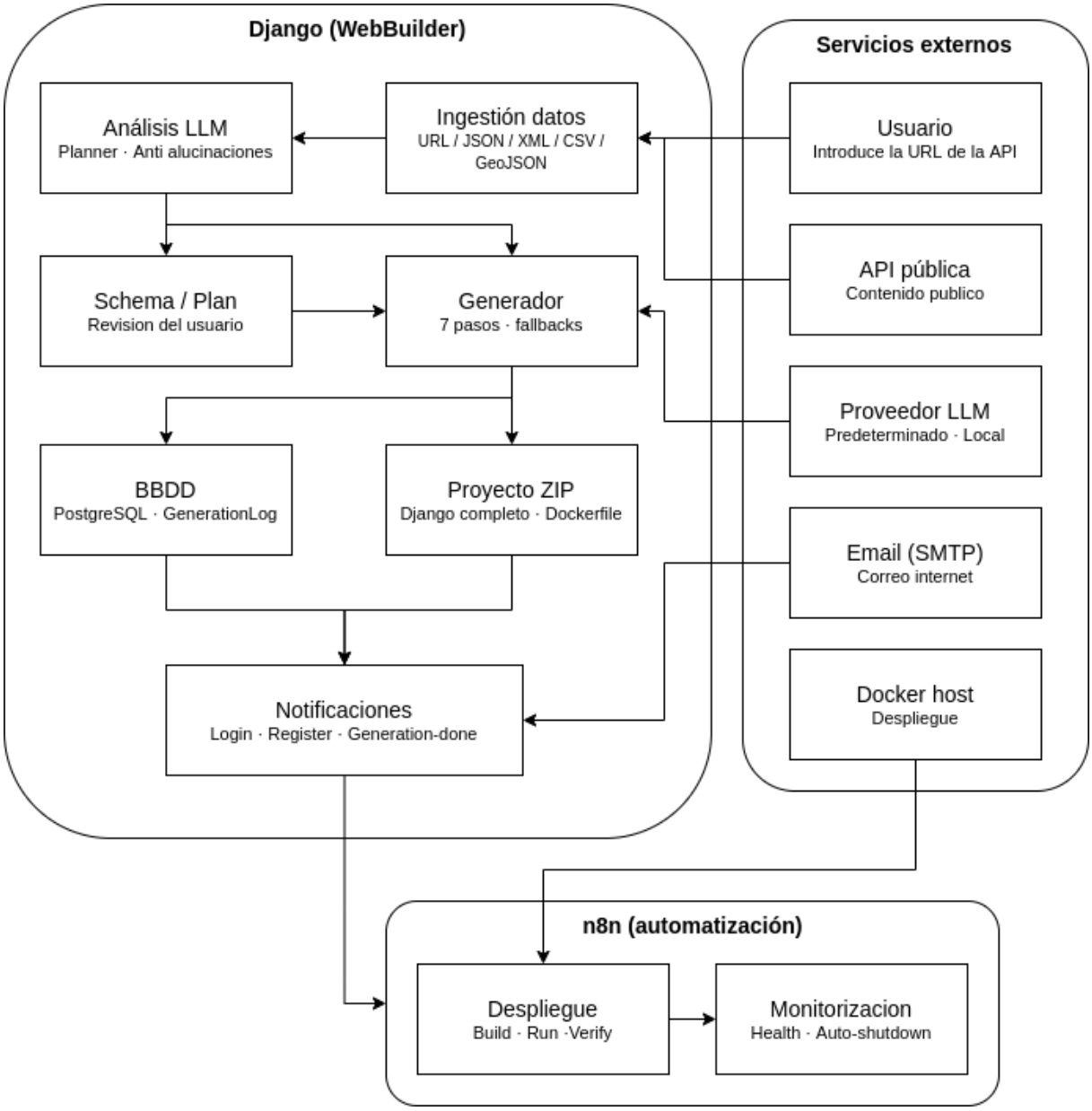

Figura 4.1: Arquitectura general del sistema WebBuilder

4.2. Fase 1: Definición del proyecto

El punto de partida del proyecto fue una reunión con el tutor en abril de 2025, en la que le comenté posibles ideas de trabajo. En esa primera toma de contacto surgieron un par de ideas más, las cuales finalmente se acabaron descartando en nuestra segunda reunión, ya que decidimos adaptarnos un poco más a la situacion actual con el tema de las paginas webs y automatizaciones.

La primera idea consistía en una aplicación web que, a partir de datos de APIs hospitalarias, ayudara a los pacientes a encontrar el centro médico más adaptado a sus necesidades según una serie de parámetros. La segunda idea ampliaba esa primera hacia un asistente de mudanza. Imaginese que alguien quiere instalarse en una ciudad, la aplicación recogería todos los datos de las APIs publicas y un parseo de los datos proporcionados por agencias y otras empresas, de tal forma que introduciendo el usuario sus requisitos, se le organizaría un posible plan de vida en la zona. Ambas ideas tenían en común la lectura y procesamiento de APIs publicas, pero el alcance era demasiado limitado, lo que limitaba el valor del proyecto.

En la segunda reunion, realizada en Julio de 2025, llegamos a una idea más acotada, una aplicacion web Django encargada de procesar los datos de cualquier API publica y generar automaticamente un sitio web personalizado. Mas tarde pensé en posibles nombres para el proyecto, hasta que al final dimos con el nombre de WebBuilder.

En ese momento aún estaba liado con los examenes, pero en los ratos libres iba recogiendo informacion y creando las posibles fases del sistema: validación de la URL, procesamiento de los datos recogidos, preferencias sobre el diseño, etc. David valoró positivamente estos pasos, lo que me dio mas energía para seguir adelante con el proyecto.

Durante los meses de verano el proyecto comenzo poco a poco creando las partes mas sencillas para ir dandole forma: la pagina de inicio, el sistema de login y registro de usuarios. Aunque en el futuro todo esto cambiaría, la base mas o menos se mantendría.

Al principio no hubo un gran desarrollo de codigo, sino que dedicamos este periodo a definir con mas detalle el flujo de la aplicacion, estuvimos investigando tecnologias y diseñando procesos que deberia seguir el sistema. Este trabajo previo fue muy importante para el correcto desarrollo guiado de la aplicacion.

4.3. Fase 2: Comienzo del desarrollo

Con la propuesta del proyecto ya definida, el desarrollo comenzó con fuerza en febrero de 2026. Poco a poco fui creando la estructura base de la aplicación: modelos, sistema de ingestion de datos, logica de analisis heuristico, y una interfaz minima y bastante pobre para poder hacer pruebas del flujo completo. La arquitectura hasta este punto era bastante sencilla: una aplicacion Django que recibia una URL, descargaba los datos de la API, los analizaba y devolvía una

propuesta de estructura para el futuro sitio web.

4.3.1. Modelo de datos

El único modelo de la base de datos en esta fase era `APIRequest`, que recogía los datos de cada petición URL realizada. Sus campos principales se muestran en la Tabla 4.1. Este modelo fue el eje central durante esta fase y será la base del análisis durante todo el proyecto.

Campo	**Tipo**	**Descripción**
url	CharField	URL de la API introducida por el usuario
raw_data	TextField	Datos en crudo descargados de la API
parsed_data	JSONField	Estructura de datos ya parseada
status	CharField	Estado del análisis: pending, processed o error
field_mapping	JSONField	Mapping resultante mostrado al usuario
created_at	DateTimeField	Fecha y hora de creación de la petición
user	ForeignKey	Usuario propietario del análisis

Cuadro 4.1: Campos del modelo APIRequest en la Fase 2

4.3.2. Ingestion de datos

El modulo de ingestion de datos, ubicado en `utils/ingest/` es el encargado de validar la URL introducida por el usuario, realizar la descarga del contenido de forma segura y detectar automaticamente el formato de los datos. La validacion comprobaba que la URL procederia de `http` o `https`, que fuese procedente de un dominio valido y que su longitud no superase los 2048 caracteres. La descarga se realizaba en modo streaming, es decir, en vez de esperar que el contenido del archivo se descargue entero, vamos leyendo trozo a trozo procesado, cada trozo tiene un limite de 1MB, y un timeout de 8 segundos. Conseguimos de esta forma evitar que una API con respuestas excesivamente grande nos tumbe o bloquee el sistema.

La deteccion de formato se realiza inspeccionando los primeros caracteres del contenido de la API, de esta forma conseguiamos distinguir entre JSON y XML, que son los contenidos manejados hasta este momento, mas adelante en el capitulo 4.7.1 se aumenta el tipo de datos a procesar. En cualquier caso siempre se devuelve una estructura Python homogenea, que se

utiliza en adelante para el sistema, de forma que no es necesario seguir trabajando con el formato original.

4.3.3. Deteccion de la colección principal

La parte mas compleja de esta fase fue entender la estructura interna de los campos de las distintas APIs. Los campos de las APIs no suelen ser una lista plana de items en la raiz, es decir, la colección principal no esta expuesta, sino que esta envuelta bajo claves de metadatos como `results`, `data` o `items`, o incluso con varios niveles de profundidad.

Para resolver esto se implemento un nuevo modulo: `utils/analysis`, donde se crearon ficheros con funciones importantes como `find_main_items`, la cual recorria recursivamente la estructura de la API, y se evaluaba con un sistema de puntuacion propio para sacar la estructura principal.

Cada lista candidata se puntuaba segun cinco factores:

- **Densidad de diccionarios**: porcentaje de elementos de la lista que son diccionarios. Las listas con menos de un 55 % de dicts recibían una penalización proporcional, ya que era poco probable que representaran una colección de entidades reales.

- **Tamaño**: listas con más items recibían mayor puntuación, aunque con un límite para evitar que una lista enorme pero trivial se impusiera sobre una colección más pequeña pero más relevante.

- **Consistencia de claves**: si todos los diccionarios de la lista compartían aproximadamente las mismas claves, era muy probable que representaran entidades del mismo tipo. Se calculaba como el promedio de claves propias de cada item sobre la unión de todas las claves observadas.

- **Bonus por claves semánticas**: si los campos de la colección incluían nombres típicos de contenido como `title`, `name`, `id`, `description` o `image`, se añadía una bonificación proporcional al número de coincidencias.

- **Penalización por profundidad**: los paths muy anidados o que pasaban por índices numéricos se penalizaban para favorecer colecciones cercanas a la raíz, que solían ser las más relevantes.

Al finalizar este scoring, se seleccionaba la lista con mayor puntuacion como coleccion principal, junto con la ruta dentro de la estructura original. En la Figura 4.2 se muestra cómo los cinco factores se combinan en la función de scoring para obtener la colección principal.

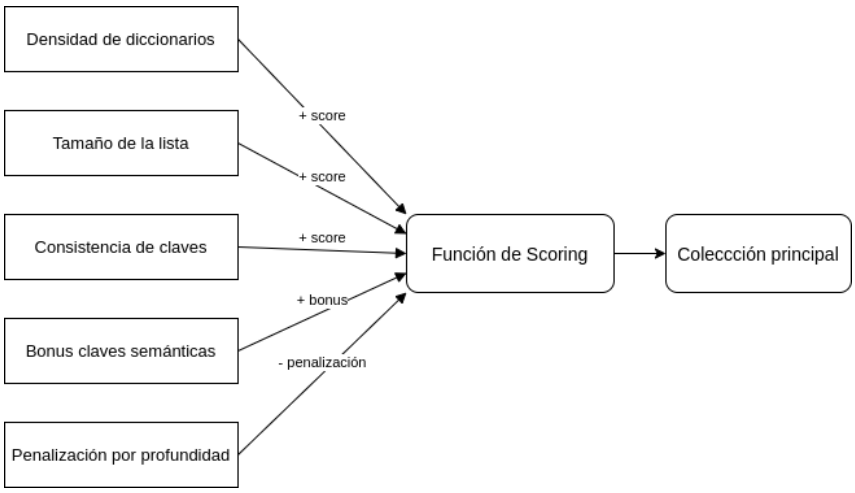

Figura 4.2: Sistema de scoring para la detección de la colección principal

4.3.4. Sugerencia automatica

Una vez localizada la coleccion principal, se realizo la implementacion del nuevo fichero **utils/mapping.py**, encargado de sugerir automaticamente que campos del dataset debian ocupar cada rol: fecha, id, descripcion, etc. Para ello se utilizaba un catalogo predefinido que asociaba cada rol con sus nombres más habituales. Por ejemplo un campo llamado `published_at`, obtenia un alto score para el rol fecha ya que contenia la subcadena `published`. Ademas de la comparacion por nombre, el sistema añadia score extra si el tipo del campo observado coincidia con el tipo esperado en el rol.

Para evitar que la misma clave del dataset se asignara como primera sugerencia de varios roles a la vez, se implementó una funcion llamada `suggest_mapping`, que aplicaba una asignación por orden de prioridad: primero se reservaba la mejor clave para el rol `id`, luego para `title`, luego para `description`, y así sucesivamente. Esto reducía la repetición de sugerencias y hacía el mapping resultante más útil para el usuario, sin embargo, ocurrian errores como que algunos campos se quedasen vacios o que al asignarse por orden de prioridad, campos sueltos se asignasen a roles que no tenian relacion.

Eleccion pagina

Para guiar mejor al usuario durante el proceso de mapping, se añadió un sistema de perfiles de intención en `intents.py`. El usuario podía indicar si quería construir un blog, un portfolio, un catálogo o un directorio, y el sistema reordenaba los roles y ajustaba cuáles eran obligatorios, recomendados u opcionales según ese tipo de sitio.

Calidad del mapping

Además, se implementó un sistema de puntuación de calidad del mapping en `mapping.py`, que evaluaba de 0 a 100 el mapping introducido por el usuario teniendo en cuenta si había asignado los roles más importantes, si existían duplicados entre roles críticos y qué campos quedaban vacíos. El resultado se mostraba al usuario con una etiqueta cualitativa: *Excelente*, *Bueno*, *Aceptable* o *Mejorable*.

Tests

Desde la primera fase se escribieron tests para los módulos principales: análisis, mapping, normalizer y parsers. Esto permitió verificar el comportamiento del sistema de scoring con datasets variados, detectando casos en los que las heurísticas fallaban antes de que llegaran al usuario.

Limitaciones del enfoque heurístico

Conforme se probaba el sistema con datasets reales muy distintos entre sí, los límites del enfoque heurístico se fueron haciendo evidentes. Una API de plantas medicinales, una de personajes de ficción y una de productos de e-commerce tienen nomenclaturas de campos completamente diferentes, y un sistema de reglas fijas no podía cubrir todos los casos con la misma fiabilidad. Cuando los nombres de los campos no coincidían con los campos de la API, el mapping era genérico y malo para el usuario. El sistema funcionaba razonablemente bien con APIs convencionales y con campos estándar, pero fallaba ante cualquier dataset con nombres o estructura poco habituales. Esta limitación fue la que motivó el cambio de enfoque descrito en la siguiente fase.

4.4. Fase 3: Integracion LLM y n8n

Con la fase 2 terminada y viendo que las limitaciones eran evidentes, decidimos hacer un cambio muy importante para el futuro del proyecto. Abandonamos la idea del sistema de reglas que teniamos, aunque no en la totalidad, ya que había partes que nos seguirían siendo útiles. Decidimos así integrar un modelo de lenguaje (LLM) como nucleo o motor del análisis y generación de las paginas. Hasta este momento el sistema intentaba entender los datos mediante puntuaciones y comparaciones, pero decidimos que fuese la Inteligencia Artificial la encargada de escoger y proponer la estructura del sitio y decidiera que campos mostrar. Como comente antes, no todo el flujo de análisis e ingesta se cambio, se eliminaron las partes de score y propuesta de dataset, pero en parseo y análisis de los campos se mantuvo y modifico levemente, se metió todo el dataset en una estructura, que seria la que se pasaria al LLM, de forma que evitariamos alucinaciones o errores del LLM en la lectura de la API. Esta parte esta mejor explicada en la subseccion 4.4.3

4.4.1. Conexion con LLM

Para poder realizar la conexion con el LLM, el primer paso fue crear el módulo **utils/llm**, y dentro creamos el archivo `client.py`, el cual implementa la funcion `chat_completions`. Esta funcion se encarga de hacer una unica cosa: enviar un mensaje al LLM y devolver la respuesta en texto plano.

Una decisión importante fue utilizar el estandar de la mayoria de proveedores para realizar la conexion, utilizando de esta forma el endpoint **/chat/completions**. En este caso creado para poder realizar pruebas con varios LLM de diferentes proveedores (OpenRouter, Groq, NVIDIA) y cualquier agente local. Otra decision fue crear `.env`, de forma que la URL base, la API key y el modelo se leen de variables de entorno definidas en dicho fichero, lo que permite cambiar de proveedor sin tocar una sola línea de código.

La función mencionada anteriormente acepta tres parámetros principales:

- **Mensaje de usuario**: la tarea concreta que se le envía al modelo en cada llamada, en este caso el prompt con los datos del dataset, las claves disponibles y los ejemplos de items reales.

- **Mensaje de sistema**: instrucciones previas que definen cómo debe comportarse el modelo antes de procesar el mensaje del usuario. Se utiliza para indicarle el rol que debe adoptar, el formato de respuesta esperado y lo que está prohibido hacer. Es opcional, ya que técnicamente todo puede ir en el mensaje de usuario, pero separarlo es una buena práctica que facilita reutilizar el mismo comportamiento base en distintas llamadas.

- **Temperatura**: parámetro que controla el grado de aleatoriedad en las respuestas del modelo. Un valor cercano a 0 produce respuestas más deterministas y predecibles, mientras que valores más altos generan respuestas más creativas pero menos consistentes. En este caso se fija por defecto en `0.2`, favoreciendo la estabilidad especialmente cuando se espera JSON estructurado como salida.

El primer proveedor utilizado fue OpenRouter, con el modelo de meta-llama: `llama-3.3-70b-instruct:free`, de acceso gratuito y potencia más que suficiente para las primeras pruebas.

4.4.2. Planner schema

Otro fichero clave fue `utils/llm/planner.py` y su funcion `generate_site_plan`. La mision era transformar el resultado del analisis en un propuesta con campos concretos de schema que el usuario pudiese aceptar y modificar a su gusto antes de continuar.

El schema tiene una forma fija que el resto del sistema espera:

- **site_type**: tipo de sitio recomendado, entre `blog`, `portfolio`, `catalog`, `dashboard` u `other`.

- **site_title**: nombre descriptivo del sitio propuesto por el modelo.

- **fields**: lista ordenada de campos del dataset que el sitio debe mostrar, cada uno con su clave original y una etiqueta legible para el usuario.

Para obtener este esquema, se construye un prompt estructurado que se envía al LLM. Este prompt es el resultado de una serie de pruebas con distintas formulaciones, siguiendo técnicas de *prompt engineering* [1] para conseguir respuestas consistentes y parseables. La información contenida en este prompt es la siguiente:

- **Objetivo del usuario**: texto en lenguaje natural introducido por el usuario describiendo qué tipo de sitio quiere construir.

- **Claves disponibles**: lista de campos reales extraídos del dataset durante la fase de análisis. El modelo solo puede proponer campos que estén en esta lista.

- **Ejemplos del dataset**: hasta tres items reales del dataset, para que el modelo pueda entender qué tipo de valores contiene cada campo y proponer etiquetas adecuadas.

- **Contrato JSON**: estructura exacta que debe devolver el modelo, con los campos obligatorios y su formato esperado.

- **Reglas de formato**: conjunto de instrucciones explícitas que indican al modelo que debe devolver únicamente un objeto JSON válido, sin bloques de código Markdown, sin texto introductorio y sin ningún contenido fuera del JSON.

Esta rigidez sobre el formato de salida, es intencionada, ya que tras la realizacion de varias pruebas los fallos generados por los diferentes LLM siempre eran parecidos. Como esa respuesta va a ser usada directamente, cualquier texto adicional rompe el proceso. De todas formas, debido a las alucinaciones de algunos modelos y la ignoracion de las restricciones en algunas ocasiones, se aplicaba una limpieza previa que eliminaba bloques Markdown y caracteres sobrantes del parseo JSON.

4.4.3. Proteccion anti-alucinaciones

Los errores mas comunes siempre sucedian por la misma causa, las alucinaciones del LLM. El error mas comun era inventar informacion que no existia en la API, y que nunca se en el parseo, por lo que no existe en el dataset contruido. Para ello hubo que aplicar varias capas de defensa contra alucinaciones para proteger los schemas.

- **Validación contra claves reales**: cada clave propuesta por el LLM se comprueba contra la lista de claves reales extraída del dataset. Si la clave no existe, se intenta un match sin distinguir mayúsculas y minúsculas. Si tampoco hay coincidencia, la clave se descarta silenciosamente.

- **Filtro de tokens prohibidos**: existe una lista de palabras que el modelo confunde con claves del dataset porque forman parte del propio prompt, como `available_keys`, `fields` o `examples`. Cualquier clave que coincida con esta lista se descarta automáticamente.

- **Detección de valores disfrazados de claves**: se detectan los casos en los que el modelo introduce un valor concreto del dataset donde debería haber puesto el nombre de un campo, como fechas en formato ISO, URLs completas o cadenas excesivamente largas.

- **Fallback**: si tras aplicar todas las validaciones no queda ninguna clave válida, el sistema construye automáticamente un schema de emergencia con las primeras claves del dataset, garantizando que el flujo nunca se bloquea por un fallo del modelo.

Si el parseo falla, el sistema reintenta automáticamente una vez con una pista sobre el error cometido. Si el segundo intento también falla, se activa el fallback. En la Figura 4.3 se muestra el flujo completo desde que el usuario introduce la URL hasta obtener el schema validado.

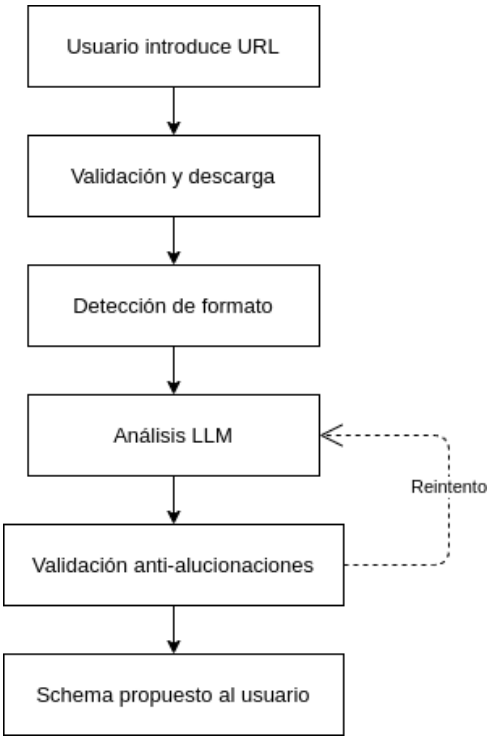

Figura 4.3: Flujo de ingestión y análisis: de URL a schema

4.4.4. Actualizacion de modelos

La integracion del LLM trajo consigo dos cambios importantes.

El primero de ellos fue una modificacion en el modelo existente **APIRequest**, añadiendole el campo `plan_accepted`. Siendo este un booleano encargado de guardar si el usuario ha aceptado el schema propuesto antes de continuar.

El segundo fue la creacion de un nuevo modelo llamado **GeneratedSite**, vinculado con el modelo APIRequest mediante una relacion `OneToOne`. Este modelo se encargaba de guardar el plan aceptado, y los templates generados para el sitio web, y un identificador unico en el campo `public_id`. Este modelo fue muy importante para la generacion de los proyectos que se desarrollaria en la siguiente fase.

4.4.5. Integracion de n8n

En paralelo a la integración del LLM, durante esta fase se incorporó n8n al sistema por primera vez. Es una herramienta de automatización de flujos que permite conectar servicios mediante webhooks y nodos, y que más adelante sería una pieza fundamental de la arquitectura del proyecto. En este momento su uso fue todavía inicial: instalar el entorno y crear el primer flujo de pruebas.

n8n se instaló en un contenedor Docker siguiendo la imagen oficial, quedando disponible en `localhost:5678`. Una vez levantado el entorno, se creó el primer flujo en n8n y se configuró el primer webhook, apuntando a la ruta `/webhook-test/WebBuilder-Register`. El prefijo `webhook-test` indica que el webhook estaba en modo de prueba, el modo que n8n ofrece para verificar que el flujo funciona correctamente antes de activarlo en producción.

La integración en Django se realizó en `views/auth.py`, donde se añadió la función `llamar_webhook`. El nodo se encargaría de notificar a un nuevo usuario en base de datos, es decir, aquellos usuarios recién registrados. Django realizaba una llamada POST a ese webhook con el nombre de usuario y el email, y n8n enviaba automáticamente un correo de bienvenida. Más adelante se completaría el flujo con el webhook de login.

La función estaba diseñada para ser completamente transparente al flujo principal: si n8n no estaba disponible o la llamada fallaba, la excepción se capturaba silenciosamente y el registro continuaba con normalidad. Esto garantizaba que un fallo en el sistema de notificaciones nunca

pudiera bloquear el acceso a la aplicación.

4.5. Fase 4: Generador de proyectos Django

Tras la implementacion del LLM y el sistema de schema funcionando correctamente, el siguiente paso era el mas complicado del proyecto: convertir el schema aprobado por el usuario en un proyecto django completo, funcional y desplegable. Durante esta fase del proyecto surgió el fichero más importante hasta la fecha: **utils/generator/project_generator**, el nucleo de WebBuilder.

4.5.1. Arquitectura del generador

El generador se organiza como un orquestador de siete pasos secuenciales, cada uno con su responsabilidad concreta y encargado de producir uno o varios ficheros del proyecto final. El LLM interviene en los primeros seis pasos, que se encargan de la creacion de ficheros, pero en el septimo paso ya no interviene, y se concluye la ejecución con el ensamblaje de todos los ficheros generados anteriormente.

- **Paso 1 — Estructura de páginas**: el LLM decide qué páginas tendrá el sitio, sus URLs, sus nombres de vista y si cada página es un listado, una página de detalle o una portada. El resultado es una lista de páginas que guiará todos los pasos siguientes.

- **Paso 2 — models.py**: el LLM genera el modelo de datos Django adaptado a los campos del dataset.

- **Paso 3 — views.py y urls.py**: el LLM genera las vistas necesarias para cada página decidida en el paso 1. El fichero `urls.py` se genera en Python directamente a partir de la lista de páginas, sin intervención del modelo.

- **Paso 4 — base.html**: el LLM genera la plantilla base del sitio, con la estructura HTML común a todas las páginas: cabecera, navegación y bloque de contenido.

- **Paso 5 — Templates por página**: el LLM genera un template HTML independiente para cada página, adaptado a su tipo y a los campos del dataset.

- **Paso 6 — load_data.py**: el LLM genera un comando de gestión de Django que, al ejecutarse, descarga los datos de la API original y los inserta en la base de datos del sitio generado. Este fichero es fundamental para que el sitio tenga contenido real desde el primer arranque.

- **Paso 7 — Archivos fijos**: se ensamblan todos los ficheros de infraestructura que no dependen del dataset ni del LLM: `settings.py`, `manage.py`, `wsgi.py`, `requirements.txt`, `Dockerfile` y `entrypoint.sh`.

El resultado final es un diccionario Python donde cada clave es una ruta relativa de fichero y cada valor es su contenido. Este diccionario se guarda en el campo `project_files` del modelo `GeneratedSite`, comentado mas adelante en la subseccion 4.5.3

En la Figura 4.4 se muestra la secuencia completa de los siete pasos del generador.

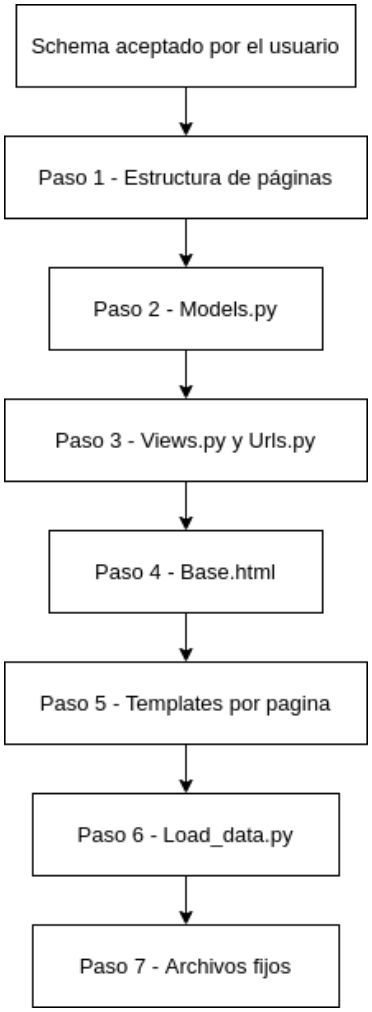

Figura 4.4: Los 7 pasos del generador de proyectos Django

4.5.2. Fallbacks generacion

Durante las pruebas de generacion aprendí que las alucinaciones del LLM eran constantes y que iba a llevar mucho trabajo pulir cada prompt de generacion. El LLM cometia fallos en cualquiera de los pasos, devolviendo codigo vacio, con errores de sintaxis o simplemente codigo random no adaptado a lo que tocaba en ese paso. Para poder tener un proyecto funcional aunque con algun error, decidí añadir **fallbacks** por cada template generado.

La funcion de los fallbacks era sencilla, si el proyecto tenia un fallo grave, el fallback actuaba y generaba una version minima pero funcional del fichero en cuestión, de esta forma el proyecto saldría funcional sin fallos graves, existirían fallos mínimos que harian que el LLM produjese un proyecto algo más genérico pero siempre desplegable.

4.5.3. Evolución del modelo de datos

El modelo `GeneratedSite` fue variando durante esta fase para reflejar los nuevos estados del sistema. Se añadieron los campos `project_files` para almacenar todos los ficheros generados, `project_name` con el nombre del proyecto, `preview_url` para guardar la URL del sitio desplegado (explicado en la siguiente subseccion 4.5.5), y dos nuevos conjuntos de estados. El primero, `generation_status`, seguía el ciclo de vida de la generación con los valores `pending`, `generating`, `ready` y `error`. El segundo, `deploy_status`, reflejaba el estado del despliegue Docker con los valores `idle`, `deploying`, `done` y `error`.

Esta separación entre el estado de generación y el estado de despliegue permitía saber con precisión en qué punto del proceso se encontraba cada sitio.

4.5.4. Dockerfile, entrypoint y requirements

Cada proyecto generado incluía un `Dockerfile`, un script `entrypoint.sh` y un fichero `requirements.txt` que en conjunto automatizaban completamente el arranque del sitio. El objetivo era que cualquier proyecto generado por WebBuilder se desplegase mediante Docker pulsando un botón, sin configuración manual por parte del usuario.

El `Dockerfile` partía de la imagen oficial `python:3.12-slim`. Copiaba el `requirements.txt`, instalaba las dependencias con `pip` y copiaba el resto del proyecto. Hasta entonces el puerto expuesto era el 8000, sobre el que se serviría la aplicación, esto variaría

más adelante con la mejora del flujo de despliegue de n8n.

El fichero `requirements.txt` incluido en cada proyecto generado contenía únicamente las dependencias mínimas para el funcionamiento del sitio: `django>=4.2,<6.0` como framework web, `requests>=2.28` para que el comando `load_data` pudiera descargar los datos de la API original, y `gunicorn>=21.0` como servidor de producción. La decisión de mantener las dependencias al mínimo fue intencionada: cuantas menos dependencias tenga el proyecto generado, menos probabilidades hay de que el `docker build` falle por incompatibilidades o paquetes no disponibles.

El script `entrypoint.sh` era el responsable del arranque completo del sitio, los pasos que ejecutaba son los siguientes:

1. **Generación de migraciones**: ejecutaba `python manage.py makemigrations siteapp` para crear los ficheros de migración a partir del `models.py` generado por el LLM, siendo siteapp el nombre de la aplicación interna del proyecto generado.

2. **Aplicación de migraciones**: ejecutaba `python manage.py migrate` para crear las tablas en la base de datos SQLite del contenedor.

3. **Carga de datos**: ejecutaba `python manage.py load_data`, el comando generado por el LLM en el paso 6 del generador, que descargaba los datos de la API original e insertaba los registros en la base de datos.

4. **Arranque del servidor**: lanzaba Gunicorn con dos workers y un timeout de 120 segundos para servir la aplicación Django en el puerto 8000.

Esta secuencia garantizaba que al acceder al sitio por primera vez ya estuviera completamente funcional: base de datos creada, datos cargados y servidor en marcha. El uso de Gunicorn en lugar del servidor de desarrollo de Django era una decisión deliberada para que los sitios generados tuvieran un nivel mínimo de robustez en producción.

4.5.5. Workflow de despligue n8n

En paralelo al desarrollo del generador de archivos, se fue construyendo el workflow mas importante del proyecto, encargado del despliegue de las paginas. La decision de realizar el

despliegue de las paginas con n8n y no con Django fue una decision mas o menos sencilla, el unico problema de desplegar con Django era que el proceso de build del Docker puede tardar unos minutos, y si se ejecutaba desde Django la web, el servidor se quedaria bloqueado durante este tiempo. Al separarlo y crearlo con n8n, Django lo unico que tiene que hacer es el enviar la señal para que comience la ejecución del workflow.

Esta señal se envia desde la ruta `/webhook/webbuilder-deploy`. Django realiza una llamada POST con el nombre del proyecto como parametro y a partir de aqui el workflow se ejecuta siguiendo los siguientes nodos sucesivamente:

1. **Descomprimir ZIP**: el primer nodo crea el directorio de despliegue en `/files/deploys/<project_name>` y descomprime el ZIP del proyecto generado en un subdirectorio `src/`.

2. **Restablecer contenedor**: antes de construir la nueva imagen, el workflow intenta detener y eliminar cualquier contenedor anterior con el mismo nombre (`wb_<project_name>`). Si el contenedor no existe, el error se ignora y el flujo continúa sin interrupciones.

3. **Elegir puerto**: el nodo escanea los puertos actualmente en uso por Docker en el rango 8100–8999 y selecciona el primer puerto libre disponible. Si no hay ningún puerto libre en ese rango, el flujo lanza un error controlado.

4. **Docker Build**: ejecuta `docker build` sobre el directorio del proyecto con un timeout de 5 minutos. Si el build falla, captura los últimos 500 caracteres de los logs de error para facilitar el diagnóstico.

5. **Docker Run**: arranca el contenedor en modo *detached* con el nombre y el puerto asignados, mapeando el puerto elegido al puerto interno 8000 del contenedor.

6. **Verificar Docker**: espera 20 segundos para dar tiempo al `entrypoint.sh` a ejecutar las migraciones y cargar los datos, y luego comprueba que el contenedor sigue en funcionamiento. Si el contenedor ha caído, recupera los últimos 40 líneas de sus logs y lanza un error con esa información para facilitar el diagnóstico.

7. **Responder al webhook**: si todo ha ido bien, devuelve a Django un JSON con la URL del preview, el nombre del contenedor y el puerto asignado, que WebBuilder almacena en el campo `preview_url` del modelo `GeneratedSite`.

Este diseño garantiza que el usuario siempre recibe una respuesta clara: si el despliegue tiene éxito obtiene la URL del sitio generado, y si falla en cualquier paso Django es notificado con información suficiente para actualizar el estado del sitio a `error` y mostrar un mensaje al usuario explicando lo ocurrido.

En la Figura 4.5 se muestra el flujo completo del workflow de despliegue.

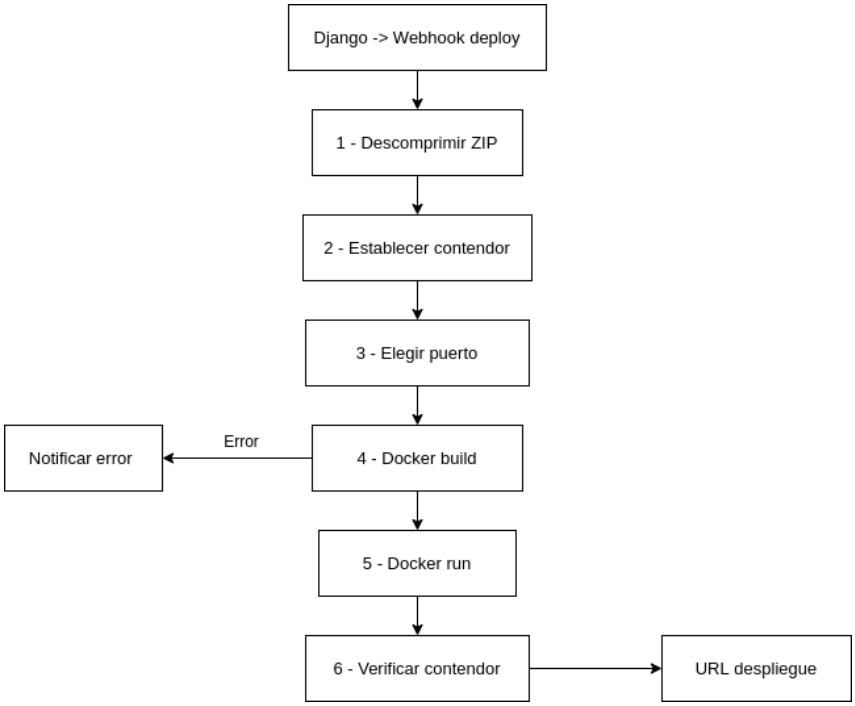

Figura 4.5: Workflow de despliegue n8n

4.6. Fase 5: Refactor del proyecto y consolidación del generador

Una vez realizadas las primeras pruebas y añadidas muchas trazas para comprobar los diferentes errores surgidos en el proceso de generacion, el codigo del fichero **project_generator.py** habia crecido demasiado y se habia convertido en un fichero dificil de depurar, y aprovechando este momento de refactorizacion se realizaron mas cambios sobre gran parte de codigos del proyecto.

Esta fase se divido en dos partes importantes. En primer lugar a la refactorizacion de partes importantes, como el fichero comentado anteriormente de mucho ficheros html y css, haciendo

su uso y modificacion futura mucho mas sencilla al tener todo divido en modulos.

4.6.1. Refactor del generador en modulos

Estos cambios fueron muy importantes, los pasos del generador se realizaban con todo el contenido dentro del mismo fichero, algo que no era de facil escalabilidad a futuro, por lo que el primer cambio fue dividir el **project_generator.py** en cuatro ficheros encargados cada uno de una labor especifica dentro de **utils/generator**, como se muestra en la Figura 4.6.

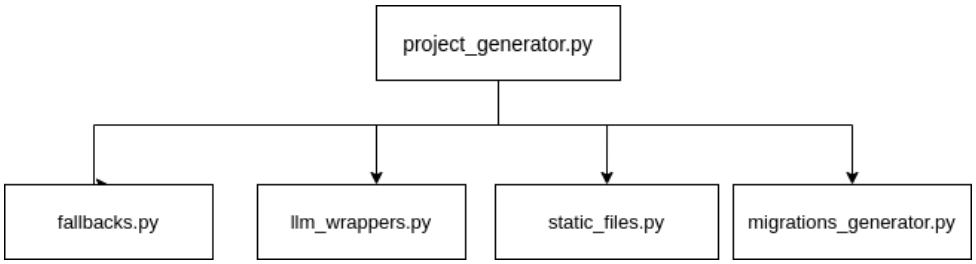

Figura 4.6: Estructura de módulos del generador tras el refactor

- **`llm_wrappers.py`**: Centralizó todas las llamadas al LLM en tres funciones reutilizables. `llm_call` para llamadas básicas que devuelven texto, `llm_json_call` para llamadas que esperan JSON como respuesta, y `llm_call_logged` para llamadas que además guardan un registro en base de datos para mejorar la depuracion. Esta ultima funcion fue muy importante en el proceso de generacion y depuracion, anteriormente si algo fallaba no habia forma de saber que habia pasado, pero de esta forma cada vez que se llamaba al LLM durante la generación de un proyecto, además de hacer la llamada normal, guardaba automáticamente en base de datos un registro con toda la información de esa llamada: el paso que se estaba generando, el prompt enviado, la respuesta recibida y si había habido algún error. Ese registro se guarda en `GenerationLog`

- **`fallbacks.py`**: Agrupó todos los fallbacks de emergencia en un único fichero. Cada función de este módulo generaba una versión mínima pero funcional de un artefacto concreto del proyecto: páginas, modelos, vistas o templates. Al tenerlos centralizados resultaba mucho más sencillo mejorarlos o ajustarlos sin tocar el orquestador principal.

- **`static_files.py`**: Extrajo toda la generación de ficheros de infraestructura que no dependen del LLM: `manage.py`, `settings.py`, `wsgi.py`, `urls.py`, `Dockerfile`,

`entrypoint.sh` y `requirements.txt`. El orquestador principal quedó así mucho más limpio, delegando en este módulo la construcción del scaffolding fijo.

- **`migrations_generator.py`**: Aisló la lógica de generación automática de la migración inicial, que parseaba el `models.py` producido por el LLM y construía el fichero `0001_initial.py` correspondiente.

4.6.2. Refactor de templates y ficheros de estilos

La misma causa que creo la decision del refactor del generador ocurrio con los ficheros de presentacion del proyecto (HTML, CSS y JavaScript). Hasta el momento los templates HTML eran ficheros que incluian todas las secciones en un unico bloque, al igual que ocurria con los ficheros CSS y JavaScript. Como comentaba anteriormente, esta fase introdujo una reorganizacion completa de la interfaz con la separacion de responsabilidades.

En cuanto a los templates se creo un nuevo modulo **partials**, encargado de contener los fragmentos que representan una seccion concreta de cada pagina. Cada fichero principal paso a contener sus partials, incluyendose mediante la etiqueta **include** de Django.

La home del proyecto, que hasta entonces era el fichero mas complejo, se dividió en seis partials independientes. En la Tabla 4.2 se muestra la estructura completa de partials tras el refactor:

Partial	**Página**	**Responsabilidad**
hero.html	home	Sección principal de bienvenida
examples.html	home	Ejemplos de sitios generados
how.html	home	Explicación del funcionamiento
llms.html	home	Modelos de lenguaje compatibles
metrics.html	home	Métricas públicas del sistema
cta.html	home	Botón de acceso a la aplicación
input.html	assistant	Formulario de introducción de URL
analysis.html	assistant	Resultado del análisis del dataset
preview.html	assistant	Schema propuesto por la IA
published_sites.html	history	Sitios generados y desplegados
recent_analysis.html	history	Análisis recientes del usuario

Cuadro 4.2: Partials HTML tras el refactor de templates

El asistente se dividió en tres partials dentro de `templates/WebBuilder/partials/assistant/`

- `input.html` — formulario del paso 1, donde el usuario introduce la URL de la API o sube un fichero local para su análisis
- `analysis.html` — resultado del paso 2, que muestra las estadísticas del dataset analizado: número de items encontrados, campos disponibles y ruta de la colección principal
- `preview.html` — resultado del paso 3, donde se muestra el schema propuesto por la IA y el usuario puede revisarlo y aprobarlo antes de continuar

El historial se dividió en dos partials dentro de `templates/WebBuilder/partials/history/`:

- `published_sites.html` — listado de sitios ya generados y desplegados por el usuario
- `recent_analysis.html` — listado de análisis recientes realizados por el usuario

Los ficheros estáticos sufrieron una reorganización equivalente. El CSS pasó a estar organizado por página y por sección dentro de `static/css/`, como se muestra en la Tabla 4.3:

Fichero	**Responsabilidad**
home/hero.css	Estilos de la sección principal de bienvenida
home/examples.css	Estilos de la sección de ejemplos
home/how.css	Estilos de la sección de funcionamiento
home/llms.css	Estilos de la sección de modelos compatibles
home/metrics.css	Estilos de la sección de métricas públicas
home/cta.css	Estilos de la sección de acceso
assistant.css	Estilos del asistente de generación
history.css	Estilos del historial del usuario
edit.css	Estilos de la página de edición del schema
login.css	Estilos de las páginas de login y registro
navbar.css	Estilos de la barra de navegación
backgrounds.css	Fondos y efectos visuales compartidos

Cuadro 4.3: Ficheros CSS tras la reorganización de estilos

El JavaScript siguió el mismo criterio con ficheros independientes por página en `static/js/`, como se muestra en la Tabla 4.4:

Fichero	**Responsabilidad**
home.js	Lógica interactiva de la home
assistant.js	Lógica del flujo del asistente
+ edit.js	Lógica de la edición del schema
site_render.js	Lógica de la vista previa del sitio generado

Cuadro 4.4: Ficheros JavaScript tras la reorganización

El `base.html` se actualizó para cargar las fuentes de Google Fonts necesarias para el nuevo diseño: Inter para el texto general, y Geist y Geist Mono para elementos de código y datos técnicos.

Esta reorganización, aunque invisible para el usuario final, facilitó enormemente el mantenimiento y la evolución de la interfaz en los sprints siguientes, ya que cualquier cambio en una sección concreta de una página solo requería tocar un fichero pequeño y bien delimitado.

4.6.3. Enriquecimiento de prompts

Tras unas cuantas pruebas, llegue a la conclusion de un problema importante, podria ocurrir que un usuario no familiarizado con los prompts de IA no supiese como hacerlo adecuadamente, o que incluso lo dejase vacio. Para resolver este problema se creo el fichero `utils/llm/enrich_prompts.py`, encargado de analizar automaticamente los campos del dataset antes de construir el prompt y añadir contenido personalizado. Si el dataset tenia campos con imagenes se le indica al modelo que las usara de forma destacada. Si tenia precios, que los resalte visualmente. Si tenia fechas que las mostrara de forma clara. De esta forma se garantiza que el LLM siempre recibiera suficiente contexto para tomar decisiones razonadas. De todos modos si algo no estuviese en la mente del usuario, en el siguiente paso de editar el schema podria modificarlo añadiendo los campos necesarios o modificando los existentes.

4.6.4. Trazabilidad: GenerationLog

Anteriormente en las generaciones si algo fallaba no había forma de saber qué prompt se había enviado exactamente, qué había respondido el modelo ni si había habido reintentos. Para arreglar esto se creo el modelo **GenerationLog**, vinculado mediante una clave a su correspondiente **GeneratedSite**.

Cada llamada al LLM durante la generación de un proyecto producía un registro que almacenaba el paso concreto que se estaba generando, el modelo de LLM utilizado, el prompt de sistema y el prompt de usuario enviados, la respuesta en crudo devuelta por el modelo, los errores de consistencia detectados y si la llamada había necesitado un reintento. Esto permitía analizar a posteriori qué pasos fallaban con más frecuencia, qué modelos producían mejores resultados y qué tipos de datasets generaban más inconsistencias.

4.6.5. Verificacion de consistencia

Otro problema frecuente era que el LLM no tenia suficiente contexto entre ficheros, principal causa de las alucinaciones del LLM. Ocurria lo siguiente: se definia un campo con un nombre en **models.py** y luego en los demas ficheros se referenciaba con ligeros cambios, provocando errores de ejecución. Para detectar estos casos antes de que el usuario desplegase el proyecto, se implemento el fichero `utils/llm/consistency_checker.py`.

Este fichero extraía mediante expresiones regulares los campos definidos en `models.py` y buscaba en los demas templates las referencias a esos campos. Cualquier referencia encontrada diferente que no correspondiera al campo real del modelo, se registraba como error de consistencia. Estos errores se guardaban en el campo `consistency_errors` del modelo `GenerationLog` correspondiente, permitiendo identificar que pasos producian mas inconsistencias y como se producian, para el consiguiente arreglo a futuro.

4.6.6. Panel de métricas

Para tener un uso mas sencillo y accesible del contenido de GenerationLog añadi una vista de metricas en **views/metrics.py**, accesible unicamente para usuarios con permisos de administrador. El objetivo de esta vista era tener visibilidad real sobre el comportamiento del generador, los fallos mas recurrentes, que modelos eran mas optimos para cada tipo de pagina, etc.

El panel mostraba las siguientes estadísticas agregadas:

- **Resumen general**: número total de registros en `GenerationLog`, sitios generados y análisis realizados hasta la fecha.
- **Desglose por paso**: para cada uno de los siete pasos del generador, se mostraba el número de llamadas realizadas, cuántas habían necesitado reintento y la tasa de reintento resultante. Esto permitía identificar qué pasos eran más problemáticos y priorizar las mejoras.
- **Distribución por modelo**: agrupación de las llamadas por modelo de LLM utilizado, lo que facilitaba comparar el rendimiento de distintos proveedores cuando se probaban configuraciones diferentes.

Esta informacion resulto util para mejorar las siguientes fases donde cambios minimos producirian grandes resultados.

4.7. Fase 6: Funcionalidades avanzadas

Una vez concluida una buena base del sistema de generacion y despliegue, esta fase se dedico a implementar funcionalidades que ampliaban las capacidades del sistema en tres direcciones: historial, ingestion de datos y LLM personalizado

En primer lugar, se añadieron nuevos modelos para cubrir las necesidades que tenia el historial, se crearon `SiteVersion`, para mantener un historial completo de todas las versiones de ficheros de cada sitio generado y poder recuperar cualquier estado anterior, y `SiteUser`, para gestionar los usuarios internos de cada proyecto Django generado directamente desde WebBuilder, pudiendo asi introducir usuarios o superusuarios.

En segundo lugar, el módulo de ingestión se amplió para soportar dos nuevos formatos de entrada muy habituales en APIs de datos abiertos: CSV y GeoJSON. Detectados de forma automatica como ocurria con XML y JSON. Normalizando los datos a una lista de diccionarios (`list[dict]`) como ocurria con los anteriores tipos, para su posterior uso.

Por último, se implementó el modelo `UserProfile`, que permite a cada usuario configurar su LLM local, con su propia API key cifrada, eliminando la dependencia de seleccion cerrada de LLMs y abriendo el sistema a cualquier modelo compatible con el estándar OpenAI.

4.7.1. Nuevos formatos de entrada: CSV y GeoJSON

Hasta este momento el modulo de ingestion solo soportaba JSON y XML. En esta fase se ampliaron los formatos añadiendo soporte para CSV y GeoJSON, que son dos de los formatos mas habituales en APIs publicas.

La deteccion del formato se añadio en el archivo existente **utils/ingest/parsers.py**.

El formato GeoJSON se detectaba antes que el formato JSON, ya que es un formato mas restrictivo. Se realiza inspeccionando el campo `type` del objeto raíz: si era `FeatureCollection`, el fichero se procesaba como GeoJSON. Mientras que para el formato CSV se inspeccionaba la primera linea en busca de separadores de coma, o punto y coma, habituales en este tipo de ficheros.

El parseo de cada formato tenía su propia función:

- **CSV**: se utilizó la librería estándar `csv` de Python con detección automática del dialecto mediante `csv.Sniffer`. La lectura se limitaba a 2000 filas para evitar problemas de

memoria con ficheros muy grandes.

- **GeoJSON**: cada `Feature` del `FeatureCollection` se aplanaba extrayendo sus propiedades como un diccionario plano, añadiendo además el tipo de geometría como campo `geometry_type`. De esta forma el resto del sistema podía tratar el resultado igual que cualquier otro JSON.

4.7.2. Ejemplos few-shot por tipo de sitio

Otro problema detectado en las generaciones era que los templates HTML producidos por el LLM tendían a ser genéricos y con poco estilo visual. Para mejorar la calidad del código generado se introdujo la carpeta `utils/llm/examples/`, con ejemplos de templates HTML reales por tipo de sitio: `blog.py`, `catalog.py`, `dashboard.py` y `portfolio.py`. Estos ejemplos se incluían en el prompt del paso 5 del generador como referencia de estilo, siguiendo la técnica de *few-shot prompting* [1], que consiste en proporcionar al modelo ejemplos concretos del resultado esperado para que los imite en lugar de inventar una solución desde cero. El resultado fue una mejora notable en la calidad visual y estructural de los templates generados.

4.7.3. Versionado de proyectos generados

Hasta este momento, generar o editar una API existente en el historial, causaba que los ficheros existentes se sobreescribieran, de esta forma era imposible tener un control de versiones o recurrir a una prueba anterior que fuese mejor. Para solucionar esto se creo el modelo **SiteVersion**, enlazado mediante una clave foranea a **GeneratedSite**, como se muestra en la Tabla 4.5. Cada version estaba nombrada con un numero incremental, y almacenaba una copia completa de los ficheros, una etiqueta descriptiva opcional y la fecha de creacion del proyecto. Las versiones estaban ordenadas de forma mas reciente a mas antigua, permitiendo asi al usuario consultar un historial funcional y la posibilidad de recuperar cualquier version anterior.

Campo	Tipo	Descripción
site	ForeignKey	Sitio generado al que pertenece la versión
version_number	IntegerField	Número incremental de la versión
project_files	JSONField	Copia completa de los ficheros del proyecto
label	CharField	Etiqueta descriptiva opcional
created_at	DateTimeField	Fecha de creación de la versión

Cuadro 4.5: Campos del modelo SiteVersion

4.7.4. Usuarios del proyecto

Cada sitio Django generado incluia su propio sistema de autenticacion, son paginas de login y register. Para gestionar los usuarios de esos sitios desde la propia plataforma se creo el modelo **SiteUser**, vinculado a **GeneratedSite**, como se muestra en la Tabla 4.6. Cada usuario del sitio tenia su nombre, una contraseña y un rol. Este ultimo campo podria ser `super` para la creacion de un superusuario con acceso al panel de administracion Django del proyecto creado, o `normal` para un usuario con acceso solo al frontend. La combinacion de proyecto y nombre de usuario era unica, evitando asi duplicados dentro del mismo proyecto.

Campo	Tipo	Descripción
site	ForeignKey	Sitio generado al que pertenece el usuario
username	CharField	Nombre del usuario del sitio generado
password	CharField	Contraseña del usuario
role	CharField	Rol del usuario: super o normal

Cuadro 4.6: Campos del modelo SiteUser

4.7.5. LLM local y catalogo de modelos disponibles

Hasta este momento el modelo de LLM utilizado para generar páginas era fijo, configurado en el fichero **.env** del servidor, sin que el usuario pudiera cambiarlo. En esta fase se implementaron dos cambios importantes respecto a esto.

En primer lugar, se creó el fichero `utils/llm/llm_catalog.py`, que definía un catálogo de modelos conocidos y recomendados. Cada entrada del catálogo incluía el identificador del

modelo, su nombre legible, el proveedor, la URL base del API, una descripción y una lista de ventajas e inconvenientes. En este sprint el catálogo incluía tres modelos:

- **Llama 4 Scout** (Groq) — modelo rápido y eficiente, recomendado para generaciones ágiles con buena calidad de código
- **Llama 3.3 70B** (OpenRouter) — modelo más potente, recomendado para webs complejas que requieren mayor capacidad de razonamiento
- **Qwen3 Coder** (OpenRouter) — modelo especializado en código, especialmente bueno generando HTML y CSS limpio

En segundo lugar, se implementó un selector de modelo completo, accesible desde el inicio del proceso, justo antes de analizar la URL. El selector ofrecía tres modos de uso:

- **Predeterminado**: el sistema selecciona automáticamente el modelo más adecuado según el dataset analizado, buscando el equilibrio óptimo entre velocidad y calidad. Es la opción recomendada para la mayoría de casos.
- **Elegir modelo**: despliega el catálogo de modelos disponibles definido en `utils/llm/llm_catalog.py`, mostrando para cada modelo su nombre, proveedor, descripción y sus ventajas e inconvenientes.
- **Modelo personalizado**: utiliza el LLM configurado por el usuario en su perfil. Si el usuario no tiene ninguno configurado, esta opción aparece deshabilitada.

El catálogo cubría los casos más habituales, pero existía la necesidad de permitir que cualquier usuario pudiera conectar su propio proveedor de LLM, ya fuera uno privado, uno de pago con mejores capacidades o simplemente uno diferente a los disponibles en el catálogo. Para soportar esto se creó el modelo `UserProfile`, vinculado en relación `OneToOne` con el usuario de Django, como se muestra en la Tabla 4.7. El modelo almacenaba tres campos opcionales:

- **URL base del proveedor** (`custom_llm_base_url`): dirección del endpoint del proveedor OpenAI-compatible, por ejemplo `https://api.groq.com/openai/v1` para Groq o `https://openrouter.ai/api/v1` para OpenRouter.

- **Identificador del modelo** (`custom_llm_model`): nombre exacto del modelo tal y como lo espera el proveedor, por ejemplo `meta-llama/llama-3.3-70b-instruct:free`.

- **API key** (`custom_llm_api_key`): clave de acceso al proveedor, cifrada en base de datos mediante `EncryptedCharField` para que nunca se almacene en texto plano.

Campo	Tipo	Descripción
user	OneToOneField	Usuario de Django al que pertenece el perfil
custom_llm_base_url	CharField	URL base del proveedor OpenAI-compatible
custom_llm_model	CharField	Identificador del modelo del proveedor
custom_llm_api_key	EncryptedCharField	API key cifrada en base de datos

Cuadro 4.7: Campos del modelo UserProfile

4.8. Fase 7: Pulido y cierre

Esta ultima fase se dedico a pulir el sistema en todos sus puntos débiles, que eran: mejorar la calidad visual de los sitios generados, consolidar la infraestructura de produccion y completar los workflows de n8n que quedaban pendientes. Con infraestructura de produccion me refiero a los cambios que hacen que el proyecto sea util fuera del entorno de desarrollo, como por ejemplo la migración a una base de datos más robusta como PostgreSQL para soportar accesos concurrentes de multiples usuarios, la gestión automática de contenedores Docker activos o la monitorización diaria del sistema. Es decir, no se añadieron funcionalidades realmente nuevas, sino que se llevaron las existentes a un nivel de madurez adecuado para la entrega.

4.8.1. Sistema de presets visuales

Uno de los principales problema durante todo el desarrollo fue que los proyectos tendian a parecerse entre si visualmente, independiente del tipo de contenido de la API, del prompt del usuario y del sitio generado. Para resolverlo se creó el módulo **utils/llm/design**, organizado en tres ficheros con responsabilidades distintas. Estos ficheros eran presets.py, snippets.py y theme_rules.py.

El fichero `presets.py` define un catálogo de estilos visuales completos y muy distintos entre sí. Cada preset describe de forma exhaustiva la paleta de colores, el tipo de layout, la estética de los componentes y las palabras clave del prompt del usuario que deben activarlo. Los presets disponibles en esta fase se muestran en la Tabla 4.8:

Preset	**Estética**	**Palabras clave activadoras**
Neo Terminal	Cyberpunk, fondo negro, acento neón	terminal, dark, gaming, futurista
Bento Pop	Rejilla asimétrica, colores vivos	bento, mosaico, asimétrico
Quiet Editorial	Tipografía protagonista, estética revista	Por defecto para blogs
Magazine Split	Gran imagen dividida, tipografía impactante	Por defecto para catálogos
Glass Orbit	Glassmorphism, fondos oscuros, traslúcidos	Por defecto para portfolios

Cuadro 4.8: Presets visuales disponibles en el módulo de diseño

Cada tipo de sitio tiene asignado un preset por defecto que se aplica cuando el prompt del usuario no contiene ningun estilo descrito, garantizando que el resultado siempre tenga una identidad visual coherente con el tipo de contenido.

El fichero `snippets.py` complementa los presets con fragmentos de HTML concretos y válidos para cada estilo: cards, heros y filas de listado con las clases Tailwind exactas que corresponden a cada preset. Estos snippets se incluyen en los prompts de generación de templates como ejemplos de referencia visual, asegurando que el LLM produzca código coherente con el estilo elegido.

El fichero `theme_rules.py` centraliza tokens visuales reutilizables como clases base para contenedores, cards, botones, badges y tipografía, junto con reglas críticas de uso de color para evitar que el LLM genere clases Tailwind inválidas, como `bg-#111111` en lugar de la sintaxis correcta `bg-[#111111]`.

4.8.2. Migración a PostgreSQL

Durante todo el desarrollo, WebBuilder había utilizado SQLite como base de datos, lo cual era suficiente para el entorno de desarrollo pero inadecuado para producción. En este sprint se realizó la migración a PostgreSQL, que ofrece mayor robustez, soporte para accesos concurren-

tes y mejor rendimiento con volúmenes de datos crecientes.

La configuración en `settings.py` se actualizó para leer los parámetros de conexión desde variables de entorno: nombre de la base de datos (`DB_NAME`), usuario (`DB_USER`), contraseña (`DB_PASSWORD`), host (`DB_HOST`) y puerto (`DB_PORT`, por defecto `5432`). La configuración anterior de SQLite se migró a la nueva base de datos, teniendo los logs y métricas anteriores.

4.8.3. Workflow generation-done

Con la base de notificaciones ya preparada desde el fichero `utils/generator/notifications.py`, se implementó el workflow de n8n *generation-done*. Este workflow se activa mediante un webhook al que Django llama en cuanto termina la generación de un sitio, enviando un payload con los datos relevantes: email y nombre del usuario, título y tipo del sitio generado, URL de la API original, URL del preview si ya está desplegado, URL de descarga del ZIP, número de campos e items procesados, duración de la generación en segundos y modelo de LLM utilizado. n8n recibe este payload y envía automáticamente un correo de notificación al usuario informándole de que su sitio está listo. La llamada al webhook se realizó de forma completamente transparente al flujo principal: si n8n no estaba disponible o la llamada fallaba, la excepción se capturaba silenciosamente mediante un bloque `try/except` y la generación continuaba con normalidad.

4.8.4. Workflow health-check

El workflow de *health-check* se configuró para ejecutarse automáticamente cada día mediante un nodo *Schedule Trigger* de n8n. Su función es verificar que el sistema sigue operativo realizando una petición HTTP a la URL principal de WebBuilder y comprobando que la respuesta es correcta. Si la verificación falla, n8n envía una alerta por correo al administrador del sistema con información sobre el error detectado. Este workflow fue especialmente útil durante las últimas fases de desarrollo para detectar caídas del servidor sin necesidad de monitorización manual.

4.8.5. Workflow auto-shutdown

El workflow de *auto-shutdown* resolvió un problema operativo importante: los contenedores Docker de los sitios generados permanecían activos indefinidamente aunque nadie los estuviera usando, consumiendo recursos del servidor. Este workflow se ejecuta de forma programada cada hora, consulta la lista de contenedores activos con el prefijo `wb_` y detiene aquellos que llevan inactivos más de un tiempo configurable. Esto permite mantener el servidor limpio y los recursos disponibles para nuevas generaciones sin intervención manual.

4.8.6. Mejora del workflow de despliegue

El workflow de despliegue que se había construido en el Sprint 4 carecía de un mecanismo centralizado de gestión de errores. Si cualquiera de los nodos fallaba, la petición de Django quedaba sin respuesta. En este sprint se añadió el nodo **Responder Error**, que actúa como rama de error compartida para todos los nodos del workflow. Cuando cualquier paso falla, este nodo captura el error y devuelve a Django una respuesta JSON estructurada con el código HTTP 500, un mensaje descriptivo del error y el paso en el que ocurrió. Esto permite que Django actualice el estado del sitio a `error` y muestre un mensaje útil al usuario en lugar de dejar la operación colgada indefinidamente.

Capítulo 5

Experimentos y validación

Este capítulo se introdujo como requisito en 2019. Describe los experimentos y casos de test que tuviste que implementar para validar tus resultados. Incluye también los resultados de validación que permiten afirmar que tus resultados son correctos.

Capítulo 6

Resultados

En este capítulo se incluyen los resultados de tu trabajo fin de grado.

Si es una herramienta de análisis lo que has realizado, aquí puedes poner ejemplos de haberla utilizado para que se vea su utilidad.

Capítulo 7

Conclusiones

7.1. Consecución de objetivos

Esta sección es la sección espejo de las dos primeras del capítulo de objetivos, donde se planteaba el objetivo general y se elaboraban los específicos.

Es aquí donde hay que debatir qué se ha conseguido y qué no. Cuando algo no se ha conseguido, se ha de justificar, en términos de qué problemas se han encontrado y qué medidas se han tomado para mitigar esos problemas.

Y si has llegado hasta aquí, siempre es bueno pasarle el corrector ortográfico, que las erratas quedan fatal en la memoria final. Para eso, en Linux tenemos aspell, que se ejecuta de la siguiente manera desde la línea de *shell*:

```
aspell --lang=es_ES -c memoria.tex
```

7.2. Aplicación de lo aprendido

Aquí viene lo que has aprendido durante el Grado/Máster y que has aplicado en el TFG/TFM. Una buena idea es poner las asignaturas más relacionadas y comentar en un párrafo los conocimientos y habilidades puestos en práctica.

1. a

2. b

7.3. Lecciones aprendidas

Aquí viene lo que has aprendido en el Trabajo Fin de Grado/Máster.

1. Aquí viene uno.

2. Aquí viene otro.

7.4. Trabajos futuros

Ningún proyecto ni software se termina, así que aquí vienen ideas y funcionalidades que estaría bien tener implementadas en el futuro.

Es un apartado que sirve para dar ideas de cara a futuros TFGs/TFMs.

Apéndice A

Manual de usuario

Esto es un apéndice. Si has creado una aplicación, siempre viene bien tener un manual de usuario. Pues ponlo aquí.

Bibliografía

[1] OpenAI. Prompt engineering guide, 2024. Accedido: abril 2026.